

UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

CORSO DI LAUREA IN INGEGNERIA INFORMATICA

Security Assessment and Attack Techniques in RFID Payment Technologies

Relatore

Prof. Simone Soderi

Laureando

Giancarlo Padoan

Correlatore

Prof. Mauro Conti

ANNO ACCADEMICO 2024-2025

Data di laurea 22/07/2025

To my family, for being my constant foundation.

To my colleagues, for lightening the weight.

To my drive, for reminding me that with hard work, everything is achievable.

Abstract

As contactless payment systems become increasingly integrated into everyday life, ensuring their security is more important than ever.

Despite their convenience, RFID-based payment technologies may harbor vulnerabilities that can be exploited by attackers. This thesis presents a practical security assessment of Mifare Classic 1K tags, which are still widely used in legacy systems. Using the Proxmark3 device, a full tag cloning process was executed, enabling both double-spending and replay attacks, and exposing critical weaknesses in the proprietary CRYPTO1 cryptographic protocol.

The analysis confirms that real-world attacks are not only feasible but also relatively accessible. Furthermore, the study demonstrates that even newer counterfeit-resistant mechanisms can be effectively bypassed.

The thesis concludes by discussing possible countermeasures and design improvements to enhance the resilience of legacy RFID-based payment systems and better protect future deployments.

Sommario

Con l'integrazione sempre più diffusa dei sistemi di pagamento contactless nella vita quotidiana, garantire la loro sicurezza è diventato fondamentale.

Nonostante la loro comodità, le tecnologie di pagamento basate su RFID possono presentare vulnerabilità sfruttabili da potenziali attaccanti. Questa tesi presenta una valutazione pratica della sicurezza dei tag MIFARE Classic 1K, ancora ampiamente utilizzati in sistemi legacy. Utilizzando il dispositivo Proxmark3, è stato eseguito un processo completo di clonazione del tag, che ha permesso la realizzazione di attacchi di double spending e replay, evidenziando criticità significative nel protocollo crittografico proprietario CRYPTO1.

L'analisi conferma che attacchi reali non solo sono fattibili, ma risultano anche relativamente accessibili. Inoltre, lo studio dimostra che perfino meccanismi più recenti di protezione contro la contraffazione possono essere efficacemente elusi.

La tesi si conclude con una discussione su possibili contromisure e proposte di miglioramento progettuale volte a rafforzare la resistenza dei sistemi di pagamento RFID legacy e a garantire una maggiore sicurezza nelle implementazioni future.

Contents

1	Introduction	1
2	Background information	3
2.1	RFID Technology	3
2.1.1	Tag RFID Near-field	4
2.1.2	Security on RFID devices	5
2.2	Cryptography concepts	6
2.2.1	Linear Feedback Shift Register	6
2.2.2	Authentication Mechanism	7
2.2.3	Anti-Collision & Application Selection	7
3	Mifare Classic 1K	11
3.1	Memory Layout	11
3.1.1	AC bits	13
3.1.2	Types of block	13
3.2	CRYPTO1 Stream Cipher	15
3.2.1	Authentication Mechanism	15
3.2.2	Data Encryption Phase	16
3.3	MIFARE Classic Vulnerabilities	17
3.3.1	Parity Bits Weakness	17
3.3.2	Nested authentications	18
4	Devices & Software	19
4.1	Proxmark3 RDV4.01	19
4.2	ProxSpace	19
4.3	Proprietary Key-tag RFID	20
4.4	Magic Mifare 1K key-tag	20
4.4.1	Gen1a Magic	21
4.4.2	Gen4 Ultimate Magic	21

5	Cloning Attack	23
5.1	Attack Implementation	24
5.1.1	Recon Phase	24
5.1.2	Data gathering Phase	25
5.1.3	Weaponization Phase	28
5.1.4	Delivery Phase	32
5.2	Motivation behinds the attack	36
6	Security Analysis and Mitigations	39
6.1	Results	39
6.2	Attack Complexity	40
6.3	Economic Impact	40
6.4	Possible Mitigation Strategies	40
7	Conclusion	43
	Bibliography	46

List of Figures

2.1	RFID Near-field Power&Data Exchange Mechanism.	4
2.2	LFSR implemented on CRYPTO1, Mifare 1K cipher algorithm.	6
2.3	Tag selection procedures.	8
2.4	MIFARE Standard application selection.	9
3.1	Memory Layout of a Mifare Classic 1K. (Source: NXP Semiconductors [15].) .	12
3.2	Layout of a Mifare Classic 1K sector.	12
3.3	Manufacturer block with a 4-byte NUID. (Source: NXP Semiconductors [15].)	12
3.4	Manufacturer block with a 7-byte UID. (Source: NXP Semiconductors [15].) .	13
3.5	Access Condition. (Source: NXP Semiconductors [15].)	14
3.6	Data format in a Value block. Source: NXP Semiconductors[15].	14
3.7	Mifare Classic Authentication Mechanism.	15
3.8	CRYPTO1 Cipher.	17
3.9	Authentication & Data Encryption Phase on CRYPTO1 Cipher.	17
4.1	Proxmark3 RDV4.01 Reader.	20
4.2	Gen1a & Gen4 Magic Tag used.	21
4.3	Backdoor block on a <i>Gen4</i> magic tag.	22
4.4	Magic capabilities status bits on backdoor block of <i>Gen4</i> magic tag.	22
4.5	Shadow mode status bits on backdoor block of <i>Gen4</i> magic tag.	22
5.1	Attack flow diagram.	23
5.2	Visual representation of the attacker in the recon phase.	24
5.3	<i>hfsearch</i> response.	24
5.4	Visual representation of the attacker in the data gathering phase.	25
5.5	<i>hf mf chk</i> result.	26
5.6	<i>hf mf autopwn</i> result.	27
5.7	<i>hf mf dump</i> result.	28
5.8	Visual representation of the attacker in the weaponization phase.	28
5.9	<i>hf mf search</i> result.	29

5.10	<i>hf mfcload -f namefile-dump.bin & hf mfcview</i> result.	30
5.11	<i>hf mfssearch</i> result.	30
5.12	<i>Backdoor memory</i> setup command.	31
5.13	<i>Dump load</i> & Backdoor memory reset.	32
5.14	<i>hf mfcview</i> error.	32
5.15	Visual representation of the attacker in the delivery phase.	33
5.16	All vending machine's status lights.	33
5.17	Gen1a magic tag with corrupted memory deployed.	34
5.18	<i>hf mfdump</i> after the delivery phase.	34
5.19	Gen1a magic tag with poisoned UID deployed.	35
5.20	Gen4 magic tag before and after a transaction.	36
6.1	Manufacturer Block on Mifare Classic 1K.	39

Chapter 1

Introduction

RFID (Radio Frequency IDentification) [1] technology has become an integral part of modern life, enabling a wide range of applications in sectors such as public transportation, building access control, inventory management, and contactless payment systems. Its low cost, ease of integration, and convenience have contributed to widespread adoption across both critical infrastructure and commercial domains [2][3][4].

However, these same advantages come with significant security challenges. Many RFID systems, particularly older ones, were not designed with robust security in mind. As a result, they are vulnerable to various forms of attack, ranging from data interception to tag cloning and unauthorized access. One of the most notable examples is the *Mifare Classic 1K* [5], a widely deployed RFID tag known for its weak cryptographic protections and vulnerability to cloning attacks [6].

Despite the availability of more secure alternatives, *Mifare Classic* tags continue to be used in critical environments, including legacy payment systems and public vending machines, due to their low replacement cost and broad existing infrastructure. This ongoing usage poses a serious security risk, especially when the systems they support are involved in financial transactions or access control.

This thesis seeks to investigate these risks through a practical security assessment of RFID-based payment solutions, with a particular focus on the *Mifare Classic 1K*. The core objective is to evaluate the feasibility and real-world impact of a tag cloning attack using the Proxmark3 device, a widely used RFID analysis tool. The attack will be tested in a live scenario against a vending machine system that relies on *Mifare Classic* for payment validation.

Beyond demonstrating the practical execution of such an attack, the research also explores the broader implications of these vulnerabilities. By analyzing the effectiveness of counterfeit-capable tags [7] [8] and assessing multiple tag generations, the study highlights the risks associated with outdated RFID technologies. Finally, the thesis proposes concrete mitigation strategies

and countermeasures aimed at improving the security of both existing and future RFID-based payment systems.

Chapter 2

Background information

2.1 RFID Technology

The RFID technology, acronym of *Radio Frequency Identifier*, is a way to identify people, animals, and objects. This mode is based on data transmission techniques that use inductive coupling and electromagnetic waves [2][9].

There are two main types of RFID devices: *active* e *passive*.

- *Active* RFID devices are equipped with an internal power source, such as a battery, which allows them to transmit signals over long distances, up to hundreds of meters [3].

- *Passive* RFID devices, which are of interest to us, do not have an internal power source. Instead, they obtain the energy required to operate directly from the electromagnetic field generated by an RFID reader [3].

The mechanism through which power is transferred to a passive tag depends on the operating frequency and physical distance from the RFID reader. Two distinct electromagnetic phenomena are involved:

- *Inductive coupling*, which occurs on the *near-field* region, defined as the area close to the antenna, relies on magnetic fields and is typically used in low-frequency (LF, ca. 125MHz) and high-frequency (HF, ca. 13.56 MHz) RFID systems [1]. This is the case of *Mifare Classic* tags.
- *Electromagnetic wave propagation*, relevant in the *far-field*, is based on capturing radiated EM waves and is used in ultra-high frequency (UHF, ca. 860-960 MHz) RFID systems, which enables longer read ranges [10].

2.1.1 Tag RFID Near-field

Faraday's law of electromagnetic induction, described in (2.1), is the foundation on which the technology for *near-field* exchanging energy and data between a reader and an RFID tag is developed [3].

$$\mathcal{E} = -\frac{d\Phi_B}{dt} \quad (2.1)$$

On Equation (2.1), $\mathcal{E}$ is the induced electromagnetic force, N is the number of turns in the coil, and $\frac{d\Phi_B}{dt}$ is the variation of the magnetic flux [1].

The RFID reader generates an Alternating Current (AC) through a coil, such that an electromagnetic field is generated in the immediate surrounding region (typically 15 cm). When a passive RFID tag, with its own small coil, enters this magnetic field, a voltage is induced in the coil. Using capacitors, the induced voltage can be rectified, stored, and used to power the tag's internal circuit.

The tag modulates the load on its own coil, causing changes in the magnetic field. These changes propagate back as variations in the reflected impedance, and, detected by the reader, are interpreted as transmitted data through appropriate *load modulation* techniques [1].

These load modulation techniques are specified in the *ISO/IEC 14443 Type A standard* [11], which defines the physical and data link layers used by many contactless smart cards, including MIFARE Classic.

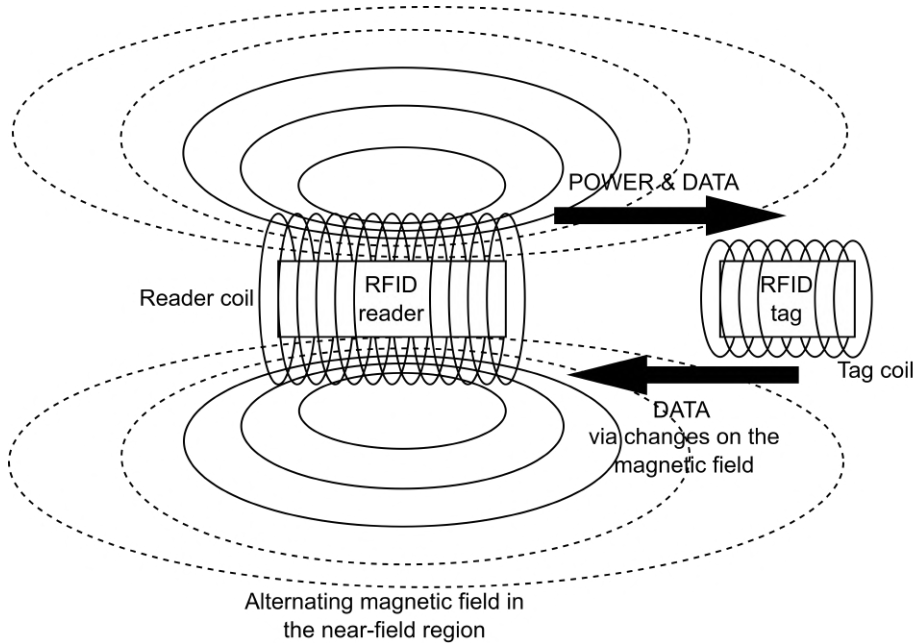


Figure 2.1: RFID Near-field Power&Data Exchange Mechanism.

The difference between *near-field* and *far-field* RFID is not only about distance or field range, but reflects the two different physical principles, inductive coupling and electromagnetic

wave propagation, governing energy transfer and signal communication [1]. Recognizing this distinction is essential when asserting security, as the nature of the coupling impacts the types of attacks that are possible, the required tools, and the level of accessibility that an attacker might have.

As RFID systems become increasingly integrated into critical domains such as public transportation, logistics, and payment, ensuring their security, also in legacy systems that use Mifare Classic, is no longer a theoretical concern but a real-world necessity [3]. The next section explores some well-known categories of threats, and their implications for systems that process untrusted tag data.

2.1.2 Security on RFID devices

Despite its convenience, RFID technology introduces significant security vulnerabilities. Because data is transmitted wirelessly and often in plain text, unauthorized interception or data manipulation is relatively easy without proper protection methods implemented [1]. Great improvements have been made in the development of cryptographic codes and transmission protocols to overcome this problem [12].

There are three main types of RFID malware: *RFID Exploit*, *RFID Worm*, *RFID Virus* [13][4].

RFID Exploit

An RFID exploit refers to an attack in which maliciously crafted data is tampered with on an RFID tag, to compromise the backend system of the reader. Unlike low-level hardware attacks, RFID exploits take advantage of vulnerabilities in the processing system of the data.

Typical techniques include:

- *SQL Injection*, inserting SQL command in the tag's data, that inserted in the reader input field and processed to compromise the functionality of the database [14].
- *Buffer Overflow*, writing over-sized data in the input buffer of the reader to trick the system into the execution of arbitrary codes [14].
- *Unauthorized memory access*, writing over-sized data in the input buffer of the reader to compromise memory's areas otherwise not accessible [14].

RFID Worms

RFID worms are self-replicating malware that spreads automatically through RFID systems with write capabilities, using network-connected infrastructure.

The process begins when an infected RFID tag is scanned by a reader that has reading and writing capabilities with no protection. The infected reader now injects the malicious code into

the tags it writes. Those tags, when scanned by another reader, infect new systems, iterating the process[14][4][13].

RFID Virus

RFID Virus are like worms, but these do not require a network connection to spread and infect new systems. To propagate, *RFID Virus* needs physical interaction [4][13]. A working proof of concept of such RFID-based malware infections is demonstrated in [14].

2.2 Cryptography concepts

To understand the vulnerabilities exploited in this thesis, it is important to review some foundational cryptographic mechanisms used in RFID authentication systems. In this thesis, the *MIFARE Classic 1K* [15] tags are of our interest, and their legacy technology relies on lightweight cryptographic primitives, nowadays exploited [16].

Modern RFID solutions now implement standardized and robust cryptography (e.g., AES), but *MIFARE Classic* relies on two custom mechanisms, that are a 48-bit Linear Feedback Shift Register (LFSR) and a basic challenge-response authentication scheme [15].

These elements are crucial to understanding both how the system works and how it can be attacked.

2.2.1 Linear Feedback Shift Register

A *Linear Feedback Shift Register*, often abbreviated as LFSR, is a sequential shift register circuit, often hardware-integrated, that allows pseudo-random bit sequences to be generated.

It consists of an N-bit register where bits are shifted to the right at each clock cycle. A feedback function, typically using XOR operation on selected *tap* positions, determines the new input bit that is inserted at the leftmost position [17][18][16].

As introduced in Section 2.2, in *MIFARE Classic 1K*, the CRYPTO1 cipher uses a 48-bit LFSR as its core keystream generator, making the cipher fast and compact, but also introducing weaknesses that can be exploited [15]. See Section 3.3.

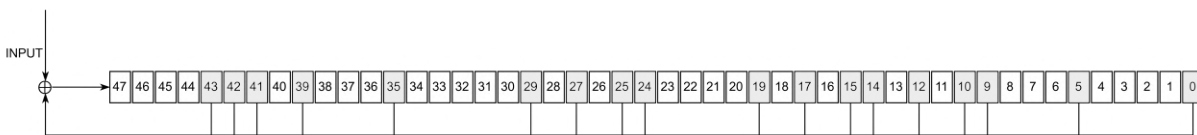


Figure 2.2: LFSR implemented on CRYPTO1, Mifare 1K cipher algorithm.

2.2.2 Authentication Mechanism

In RFID systems, authentication can be implemented in various modes, ranging from simple static identifiers to more secure dynamic protocols.

These methods have all the same goal; they aim to verify the identity of a tag to a reader, or vice versa.

Below are three principal authentication mechanisms utilized in RFID-based systems:

- *Password verification*, this is the most basic form of authentication where the tag stores a static password, and the reader has to provide the correct password to gain access. If the password matches, the tag allows the communication.

With the ease of implementation, this method suffers from significant vulnerabilities such as, password transmitted often in plain-text, making interceptable via eavesdropping; password reuse to perform replay attacks; and unauthorized cloning knowing of tags [14][19].

- *Shared-Key Authentication*, in this method both the reader and the tag stores a common secret key. During the authentication process, the reader may send an encrypted message or request that the tag must decrypt or compute using the shared key. This establish mutual trust.

The limitations of this method are that if the key is retrieved from one device, all other ones sharing the same key are compromised and also in large deployments key management becomes challenging [20][21].

- *Challenge-Response*, in this method the verifier sends the prover a challenge, usually a nonce or a random value, and the prover computes a response.

To compute the response, the prover usually combines the challenge with a secret key and hashes the result, or it gives it as input to a secret algorithm.

At the verifier side, the response is checked to establish that the prover knows the secret, and the communication can start [22].

In the context of this thesis, the *MIFARE Classic 1K*, system uses a proprietary challenge-response protocol based on CRYPTO1 stream cipher. See Section 3.2.1.

2.2.3 Anti-Collision & Application Selection

While authentication, see Section 2.2.2, ensures that the selected tag is legitimate, it is important to understand how a specific tag is selected in the first place, and the desired application is running, especially in multi-tag environments.

The following section explains the anti-collision and application selection mechanisms in RFID-based systems.

1. Anti-collision mechanism

According to the *ISO 14443 Standard* [23], this selection process is defined at the application layer. This step is crucial because, within the reader's range, multiple tags *ISO 14443 Standard*-compliant may be present simultaneously.

The procedure shown in Figure 2.3 guarantees that only a single tag is selected. It is also guaranteed that the single tag remains selected even if multiple tags are in the reader fields.

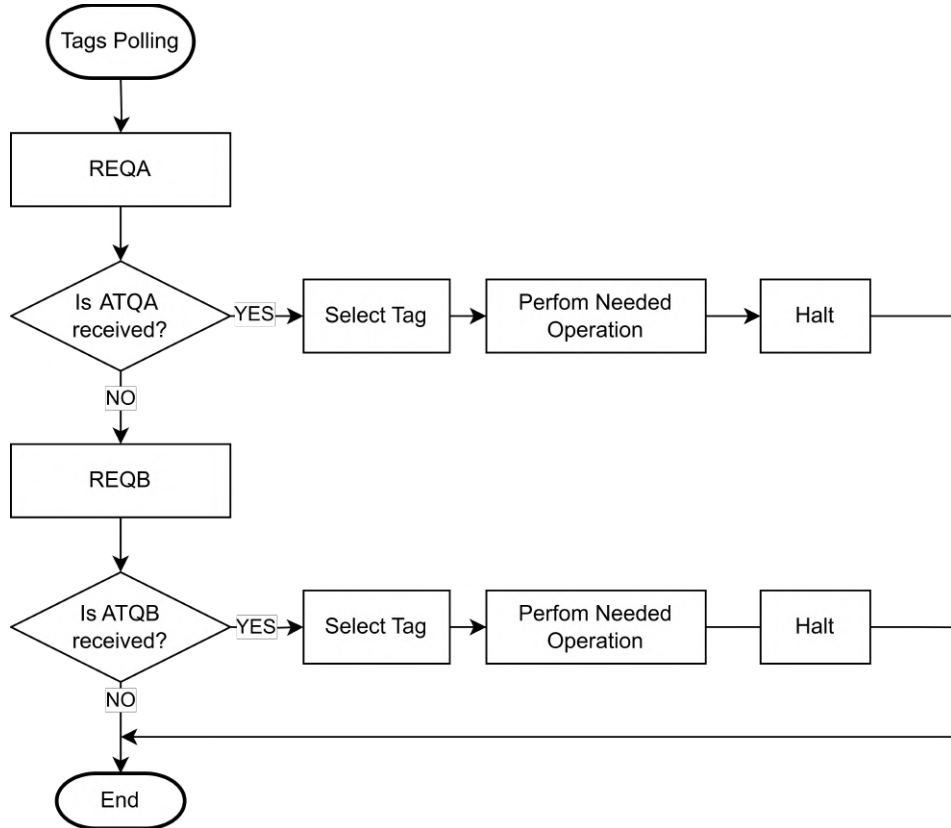


Figure 2.3: Tag selection procedures.

- REQA: Request Command, Type A = MIFARE
- REQB: Request Command, Type B
- ATQA: Answer To reQuest A, according to *IOS/IEC 14443-4* [24]
- ATQB: Answer To reQuest B
- HALT: After a successful transmission, the reader, tells the tag to go in a low-power state, to not interferer with other communication.

2. Select MIFARE Standard Application

Once a single tag has been selected for the communication, see Section 2.2.3, the reader has to select also the appropriate application on the tag. This ensures that the reader interacts with

the tag in the correct way, allowing it to handle multiple applications or properly ignore unknown unsupported applications. In the context of this thesis, in Figure 2.4, the process used for selecting a tag running MIFARE Standard application is shown.

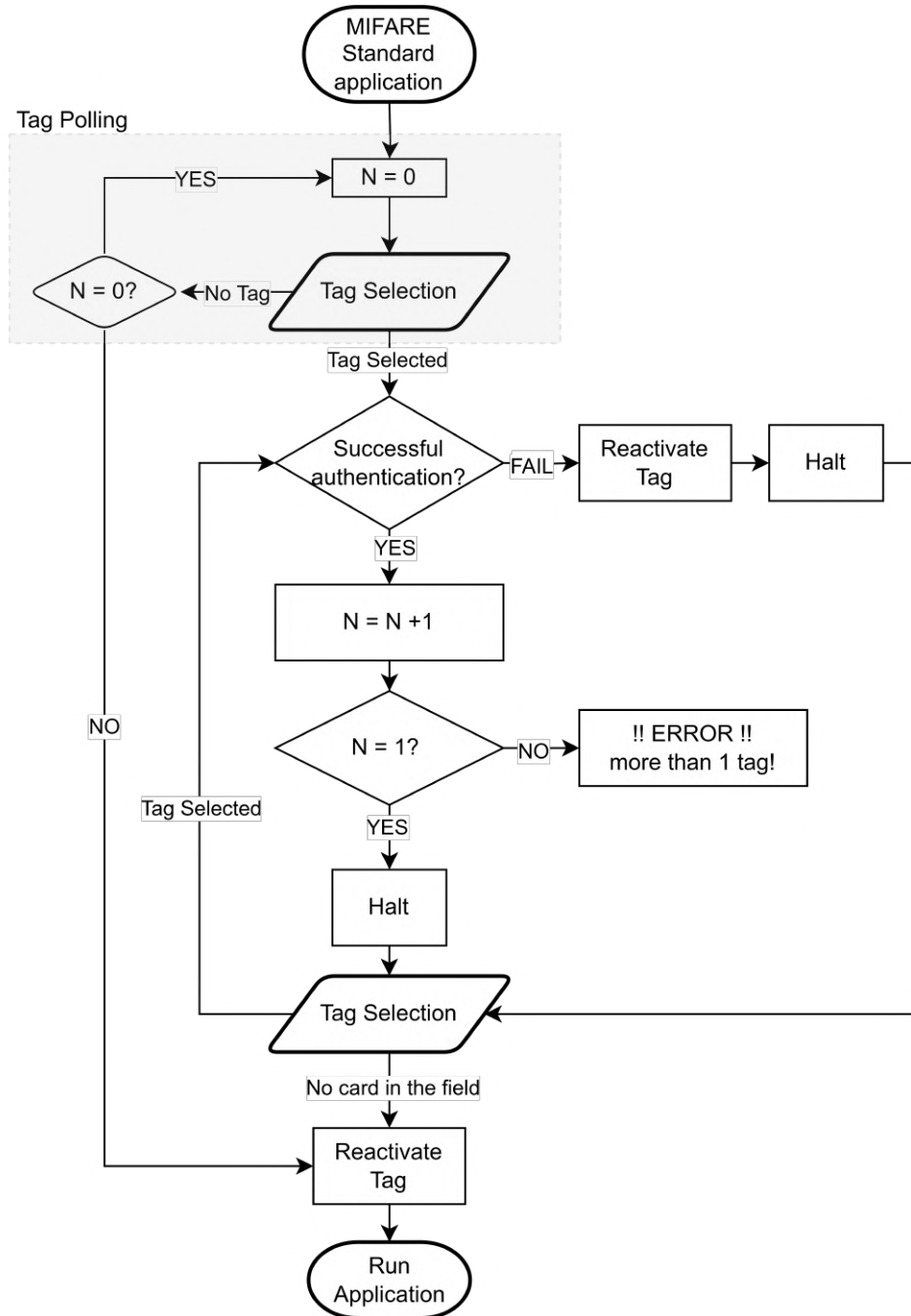


Figure 2.4: MIFARE Standard application selection.

The whole process of identifying the correct tag, establishing communication with it, and selecting the correct running application, with the respective procedures, is explained in detail in the *MIFARE ISO/IEC 14443 PICC Selection* paper [25].

Chapter 3

Mifare Classic 1K

Mifare Classic MF1S50YYX_V1, or more commonly known as *Mifare*, is a passive RFID tag, developed by NXP Semiconductors (ex Philips), which is widely used in contactless payment systems, electronic access control, public transport ticketing, and vending machines due to its low cost and simplicity [5].

3.1 Memory Layout

RFID system that rely on *MIFARE Classic 1K* chip use a well defined memory layout designed for flexible access control and secure data storage. Understanding how the memory is organized is crucial for both legitimate application development and also for identifying potential vulnerabilities.

The memory chip of *Mifare Classic 1K* tag, with a size of 1024 bytes (1 KB), is organized into 16 sectors of 64 bytes each, and each sector is further divided into 4 blocks of 16 bytes each [17][15]. See Figure 3.1.

On each sector, the last block is the *Trailer* block. In Figure 3.1, named as *Sector Trailer*, it contains the keys and bits for the access conditions and an unused *General Purpuse byte*, respectively (*Key A*, *AC bits*, *GPB*, *Key B*). If *Key B* is not used, the corresponding bits can be used as memory [17][15].

The first block of Sector 0 is the *Manufacturer block* which is usually *read-only* and can contain either a 7-byte UID or a 4-byte NUID followed by a checksum byte (BCC) obtained by XORing the UID bytes [17][15]. See Figures 3.3, 3.4.

Read, *write*, *increment*, *decrement*, *transfer* and *restore* operations can be performed in each block, following the access policies defined by the *AC bits*. Correct interpretation of these bits is crucial for determining access control policies that, if misconfigured, can represent a potential

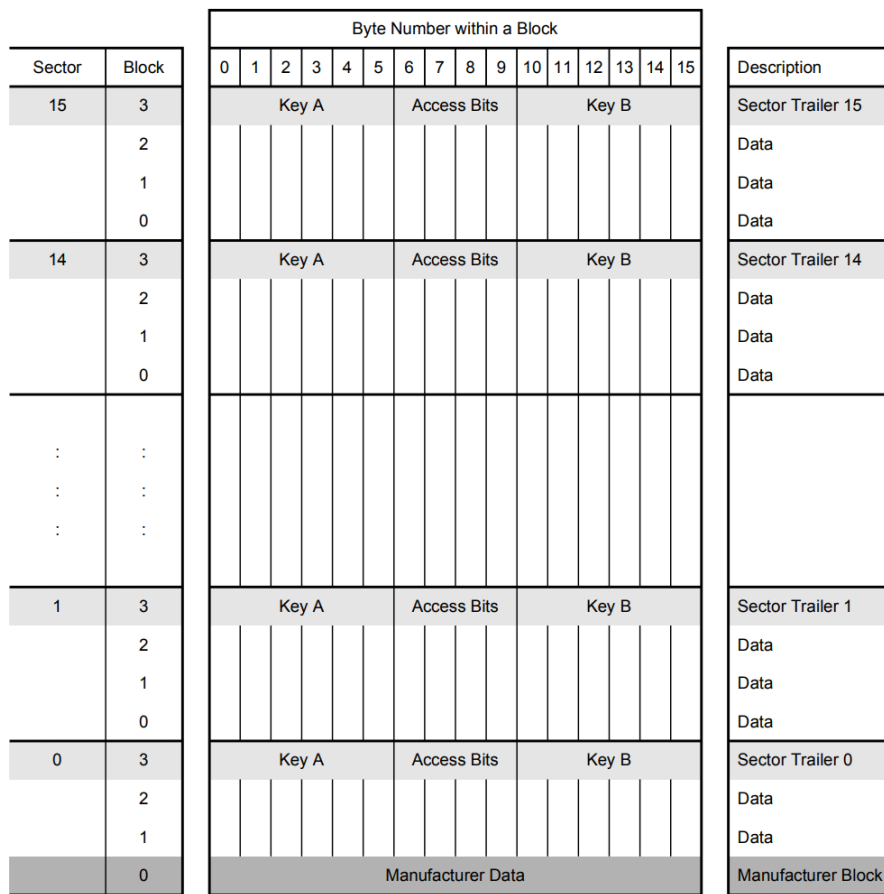


Figure 3.1: Memory Layout of a Mifare Classic 1K. (Source: NXP Semiconductors [15].)

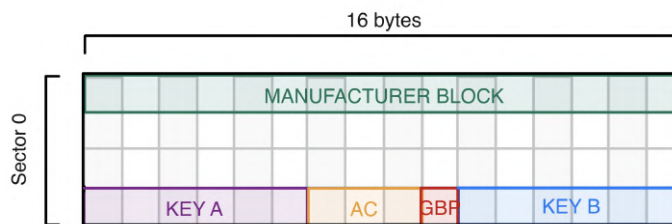


Figure 3.2: Layout of a Mifare Classic 1K sector.

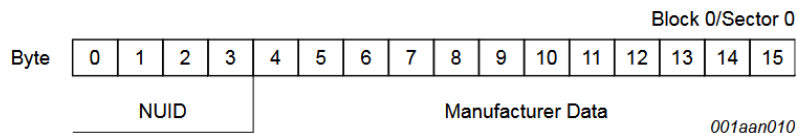


Figure 3.3: Manufacturer block with a 4-byte NUID. (Source: NXP Semiconductors [15].)

point of failure for the system.

In the context of this thesis, understanding how Access Bits work is essential, as wrong configu-

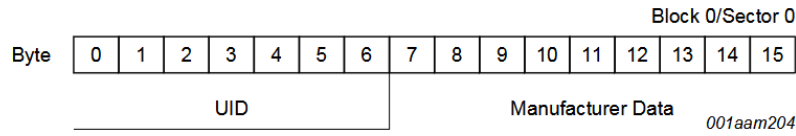


Figure 3.4: Manufacturer block with a 7-byte UID. (Source: NXP Semiconductors [15].)

ration can be exploited to bypass authentication or escalate privileges during cloning and replay attacks.

The following section explains how Access Bits define which operation is permitted on each block and explains their structure [17][15].

3.1.1 AC bits

The 3-byte AC (*Access Conditions*) defines how a sector is accessed based on the value of the keys (*Key A and Key B*).

Each triad (C1, C2, C3) of bits is stored twice, one of which is inverted, and each controls access to a single block.

This redundancy improves robustness, as any inconsistency can indicate some corruption or tampering.

Access Bits	Valid Commands		Block	Description
C1 ₃ C2 ₃ C3 ₃	read, write	→	3	sector trailer
C1 ₂ C2 ₂ C3 ₂	read, write, increment, decrement, transfer, restore	→	2	data block
C1 ₁ C2 ₁ C3 ₁	read, write, increment, decrement, transfer, restore	→	1	data block
C1 ₀ C2 ₀ C3 ₀	read, write, increment, decrement, transfer, restore	→	0	data block

Table 3.1: Access Condition. (Source: NXP Semiconductors [15].)

3.1.2 Types of block

This section describes the types of blocks in *MIFARE Classic 1K* and explains the differences between standard *Data blocks* and *Value blocks*.

Understanding this layout is essential for analyzing both the normal operation of the system and its security vulnerabilities, which are later exploited in the attack scenario presented in this thesis.

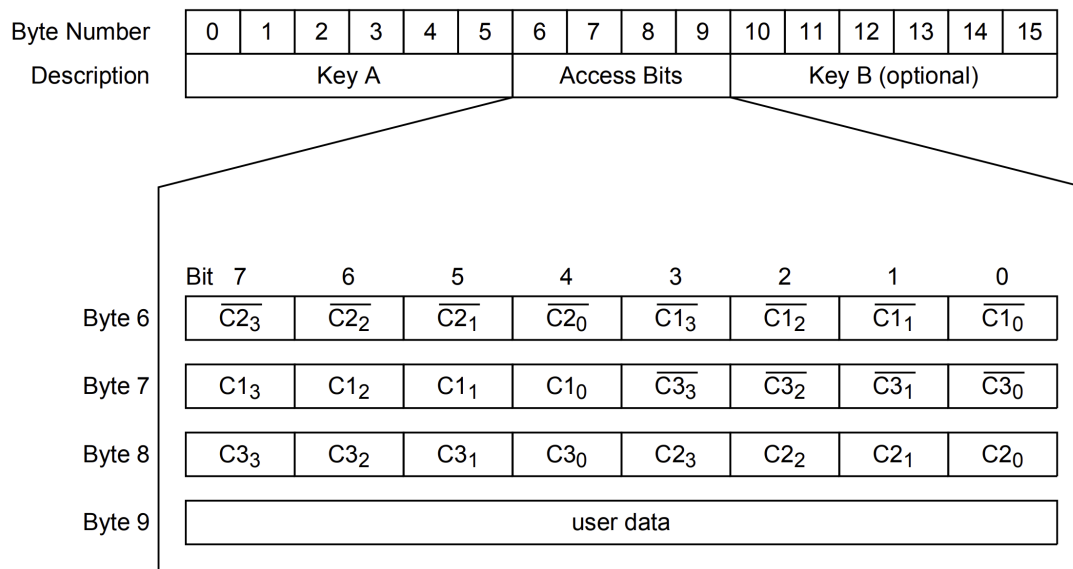


Figure 3.5: Access Condition. (Source: NXP Semiconductors [15].)

Data Block

Each sector contains three *Data blocks* of 16 bytes and one *Trailer block*, except for *Sector 0*, which includes *Manufacturer block*, two *Value blocks* and the *Trailer*.

A *Data block* can function either as a standard *Read\Write block* or as a *Value block*, depending on the configuration specified by the Access Bits [17][15].

Value Block

Value blocks allow the execution of electronic purse operations.

They have a predetermined data format; they store a 4-byte signed integer in little-endian format, if negative represented in two's complement, followed by a one-byte address (*adr*). The value is repeated in the block 3 times, one inverted and two not, to allow for error detection and correction and/or prevent physical manipulation of values, while the address is stored 4 times, two inverted and two not, and points to a backup block for the value [17][15].

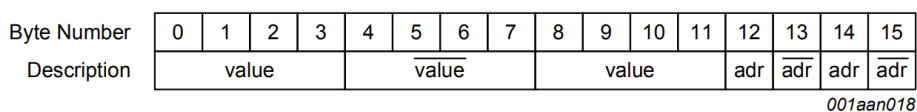


Figure 3.6: Data format in a Value block. Source: NXP Semiconductors[15].

3.2 CRYPTO1 Stream Cipher

Access control, provided by the Access Bits, alone is not sufficient to guarantee security. The communication between the reader and the tag has also to be protected against eavesdropping, replay, and manipulation.

As introduced in Section 2.2, *MIFARE Classic* uses proprietary CRYPTO1 as a stream cipher to secure both authentication and communication between the tag and the reader.

Despite being closed and not documented, CRYPTO1 has been reverse-engineered and analyzed, revealing multiple weaknesses [6][26][16][27].

The following sections explain the process that CRYPTO1 follows to secure the communication between the tag and the reader.

CRYPTO1 stream cipher operates in two phases:

1. Authentication Phase
2. Data Encryption Phase

3.2.1 Authentication Mechanism

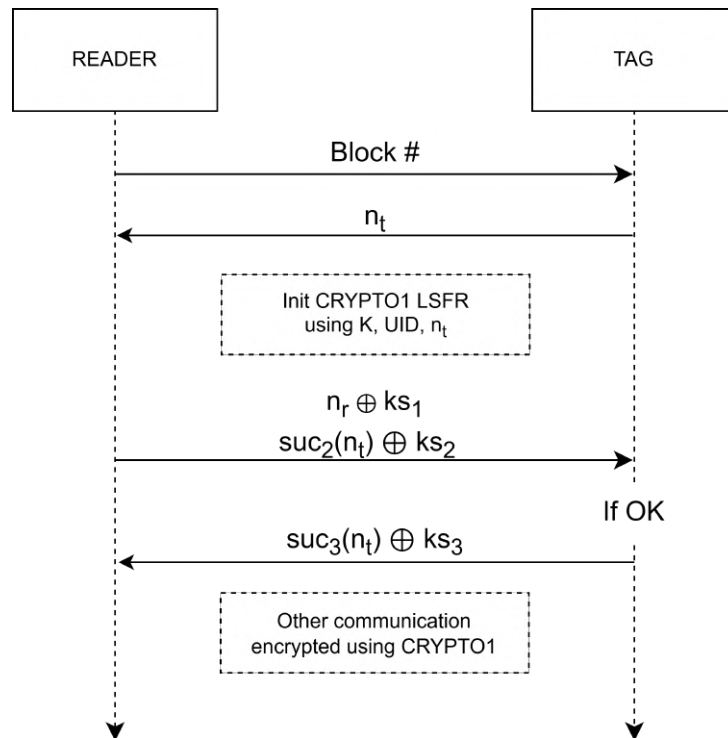


Figure 3.7: Mifare Classic Authentication Mechanism.

When a reader wants to access a sector of the card's memory, it first issues an authentication command specifying the block number. The tag then generates a 32-bit random nonce n_t and

sends it in clear.

Both reader and tag initialize their internal *CRYPTOI* LFSR state using the shared secret *key K*, the tag's *UID* and n_t [16].

The reader responds with its own 32-bit nonce n_r , encrypted under the first portion of the keystream ($n_r \oplus ks_1$), along with a proof of knowing *K* in the form of $suc_2(n_t) \oplus ks_2$.

The tag verifies that $suc_2(n_t)$ was correctly encrypted, updates its keystream state, and replies with $suc_3(n_t) \oplus ks_3$.

At that point, both sides know they share the same secret key, so all further communication is encrypted using the current *CRYPTOI* keystream.

This authentication mechanism has been studied and reverse-engineered, particularly in the works [26][6][16], which revealed significant vulnerabilities in the protocol, opening the doors for numerous attacks.

3.2.2 Data Encryption Phase

After the successful authentication phase between the reader and the tag described above, the data is encrypted *on-the-fly* using *CRYPTOI* keystream.

The encryption is based on the stream cipher model, meaning that the ciphertext is obtained by XORing plaintext and keystream.

$$ciphertext_i = plaintext_i \oplus keystream_i \quad (3.1)$$

The keystream is generated from the *i-th* internal state of the LFSR. *CRYPTOI* uses, to compute the keystream, 3 distinct non-linear feedback functions (f_a, f_b, f_c).

The feedback functions are applied on specific *taps* of the 48-bit stream.

$$\begin{aligned} y_{a1} &= f_{a1}(x_9, x_{11}, x_{13}, x_{15}) \\ y_{b1} &= f_{b1}(x_{17}, x_{19}, x_{21}, x_{23}) \\ y_{b2} &= f_{b2}(x_{25}, x_{27}, x_{29}, x_{31}) \\ y_{a2} &= f_{a2}(x_{33}, x_{35}, x_{37}, x_{39}) \\ y_{b3} &= f_{b3}(x_{41}, x_{43}, x_{45}, x_{47}) \end{aligned} \quad (3.2)$$

and the keystream is obtained by computing the last non-linear function f_c on the previously calculated values.

$$keystream = f_c(y_{a1}, y_{b1}, y_{b2}, y_{a2}, y_{b3}) \quad (3.3)$$

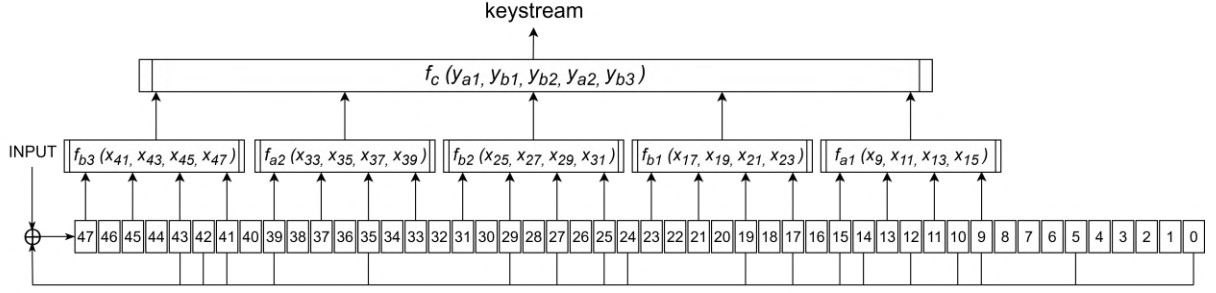


Figure 3.8: CRYPTO1 Cipher.

In Figure 3.9 a complete scheme of CRYPTO1 stream cipher with both the *Authentication Phase 3.2.1* and the *Data Encryption Phase 3.2.2* is shown.

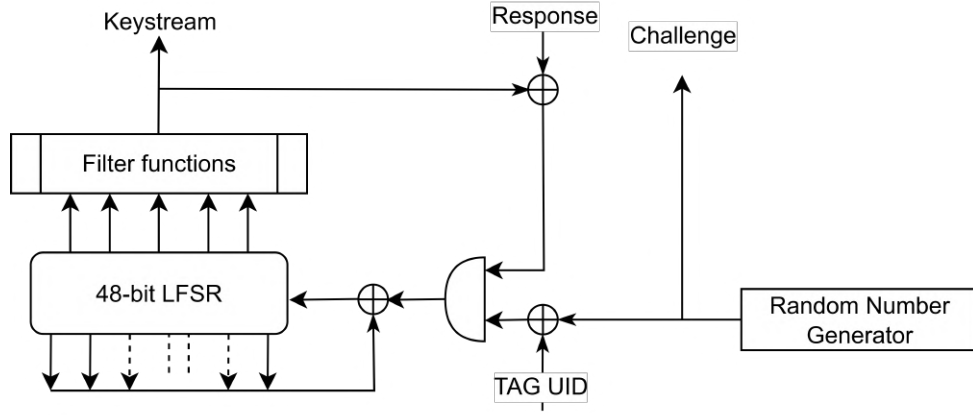


Figure 3.9: Authentication & Data Encryption Phase on CRYPTO1 Cipher.

The full functioning of CRYPTO1 cipher, including both the authentication and encryption phases, was reverse-engineered and subjected to cryptanalysis, and the most notable works on this topic are [26][6][16], which exploited structural weaknesses in the cipher; weaknesses used also in this research to succeed in the attack.

3.3 MIFARE Classic Vulnerabilities

According to [27], in this section, we introduce the vulnerabilities exploited to reverse-engineer the *CRYPTO1* algorithm and obtain the key to access the memory.

3.3.1 Parity Bits Weakness

According to *ISO14443-3* [23], for each byte of data to be sent, an even parity bit is calculated. After the 8 bits of data, the 9-th bit transmitted is the parity bit.

The major flaw is that *MIFARE Classic* parity bits are computed over the plain-text instead of the cipher-text.

In addition, the *CRYPTOI* state, and consequently, the bit used to compute the coming data bit do not change. Or rather, the $(i+1)$ -th data bit is encrypted using the same bit of keystream used to compute the i -th parity bit.

Furthermore, during the authentication protocol, when the reader sends the response $a_r = \text{suc}_2(n_t) \oplus ks_2$ to the tag's challenge and its own challenge $n_r \oplus ks_1$, the tag, before checking the answer, checks the parity bits [27].

The protocol manages four possible outcomes:

1. **At least one of the eight parity bits is wrong:** the tag does not respond.
2. **All parity bits are correct but a_r incorrect:** tag respond with code $0x5$, meaning failed to authenticate.
3. **Both all parity bits and a_r correct:** the tag respond with its answer $a_t = \text{suc}_3(n_t) \oplus ks_3$.
4. **Correct parity bits but wrong answer:** the code $0x5$ is sent encrypted, even if the reader has not authenticate itself.

3.3.2 Nested authentications

Knowing just a single key of a sector, an attacker can retrieve more keys by exploiting the following vulnerabilities.

When a reader-tag encrypted communication is already established, to gain access to a different sector, a reader has to send to the tag a new encrypted authentication command.

After the command, the internal state of *CRYPTOI* cipher is set to the corresponding key of the requested sector, and the authentication protocol described in Section 3.2.1 starts again, but this time all encrypted, including the tag challenge n_t .

Considering that the *Random Number Generator - RNG* used on *MIFARE Classic* is not truly random but based on a *LFSR Pseudo-RNG*, instead of the expected 32 bits of entropy, only 16 bits are obtained.

Following, there are only 2^{16} possible nonces, and an attacker can simply try to guess a nonce to recover the 32 bits of the keystream.

Using the information leaked through the vulnerabilities of the parity bits, explained in Section 3.3.1, an attacker can speed up the attack. Since three bits of information are leaked, there are only 2^{13} possible nonces [27].

Chapter 4

Devices & Software

To demonstrate the vulnerabilities of *MIFARE Classic* tags and complete the attack, the object of this thesis, a specific set of hardware and software tools was used. The selection was guided by their compatibility with the attacked system and their ability to perform both malicious and legitimate operations on *MIFARE Classic* tags.

This section provides an overview of the devices and software environments used during the experimental phase. Understanding the capabilities and limitations of these tools is essential to evaluate the feasibility of the attack and contextualize the results obtained.

4.1 Proxmark3 RDV4.01

In Figure 4.1 the *Proxmark3 RDV4.01* reader, a multifunctional RFID analysis device. It allows reading, writing, emulating, de/modulating, de/encoding, de/encrypting any RFID device. As we can see in Figure 4.1 the *Proxmark* is equipped with two antennas, one operating in high frequencies around 13.45 MHz and the second operating in low frequencies around 125 KHz - 134 KHz. On the second antenna it is possible to scan only a selected frequency by manually switching on $Q = 7$ and selecting the desired frequency or to do a range scan in between the two frequencies by selecting $Q = 14$ [28].

4.2 ProxSpace

ProxSpace is a *Windows-based* development environment used for compiling and flashing *Proxmark3*'s firmware. It allows using the *Proxmark3*'s CLI on Windows terminal, with no need to install a virtual machine or a dual-boot Linux. It also provides all the dependencies, like *GCC*, *Make*, *Python* needed [29].

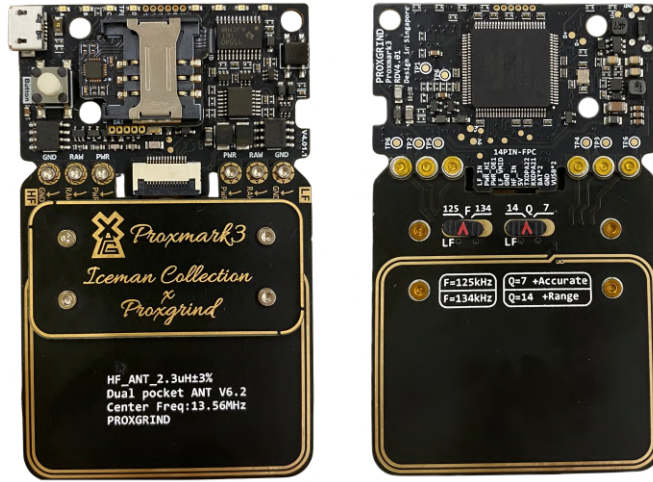


Figure 4.1: Proxmark3 RDV4.01 Reader.

4.3 Proprietary Key-tag RFID

Many vending machines and access systems adopt *MIFARE Classic 1K* tags as electronic purse systems, due to their low implementation cost and availability. The target of the attack described in this thesis is exactly this type of *MIFARE Classic*. In our scenario, the tags are proprietary, meaning that the reader and tags share pre-configured access keys, in order to enable transactions only on legit tags.

As previously discussed in Section 3.3, is possible, exploiting the vulnerabilities, to extract the secret keys enabling our attack.

4.4 Magic Mifare 1K key-tag

So-called *Magic* MIFARE tags are unauthorized clones of the above described *MIFARE Classic* tags. These *Magic* tags are developed to bypass restrictions imposed by the NXP chip. It enables the writing procedures of the Manufacturer Block (Block 0, Sector 0) and the UID, that usually are read-only and came pre-configured by the vendor. The above described characteristics are the one called *magic capabilities* on the *Magic MIFARE Classic* tags.

Particular attention is given to the fact that a wrong configuration of the manufacturer block, especially an invalid *Block Check Character - BCC*, may result in a non-usable or *bricked* tag [8]. This point is further discussed in Section 6.1.

In the literature, multiple generations of *magic MIFARE Classic*-compatible chipsets have been developed, each introducing additional functionalities. For the purpose of this attack, two versions were utilized: the first developed, *Gen1a*, and the most recent available at the time of writing, *Gen4*.

The rest of the section describes these chipsets.

4.4.1 Gen1a Magic

The *Gen1a* version is the earliest and the most widely documented and developed. The entire card can be read and written to as desired, with no restrictions [30].

Due to their diffusion, some readers have implemented procedures to detect magic cards and abort the communication with them.



Figure 4.2: Gen1a & Gen4 Magic Tag used.

4.4.2 Gen4 Ultimate Magic

The *Gen4a* is the latest and most advanced generation of *Magic tag* and comes with multiple upgrades in terms of reliability.

The tags are fully writable with *Auto BCC calculation* reducing the risk of bricking the tag.

The *Gen4* standard can be switched between *Magic mode* and *Normal mode*, by modifying a hidden backdoor block, inaccessible from a standard reader. Further explained in Section 5.1.3. This feature solves the problem of detectable magic cards.

The most interesting enhancement is that the tag can operate in *Shadow mode*, in which writes are temporarily stored in the tag's RAM, and once power is lost, the contents are restored to the previous state stored in the tag's EEPROM [31].

Gen4 Backdoor block

In Figure 4.3 is shown a configuration of the backdoor block on a *Gen4* magic tag. The block is 32-byte sized and contains information that govern the behavior of the magic tag. The bytes of our interest are:

(1-4)st-th: the first 4-byte of information enable/disable the magic wakeup function. If disabled the magic tag function like a legit tag and a reader cannot detect its magic capabilities. Setting the bytes to **7AFF** enable the magic wakeup, instead setting it to **8500** disable

```
[+] ----- GDM Gen4 Configuration -----
[+] 7AFF000000000000FF5A5A00000008
[+] 7AFF..... Magic wakeup enabled
[+] .....00..... Magic wakeup style Gen1a 40(7)/43
[+] .....000000..... unknown
[+] .....00..... Key B use allowed when readable by ACL
[+] .....00..... CUID Disabled
[+] .....00..... n/a
[+] .....FF..... MFC EV1 perso. 4B UID from Block 0
[+] .....5A..... Shadow mode enabled
[+] .....5A..... Magic auth enabled
[+] .....00..... Static encrypted nonce disabled
[+] .....00..... MFC EV1 signature disabled
[+] .....00... n/a
[+] .....08 SAK
```

Figure 4.3: Backdoor block on a *Gen4* magic tag.

it[7].

In Figure 4.4 is shown a block configuration where magic wakeup is enabled.

```
[+] ----- GDM Gen4 Configuration -----
[+] 7AFF000000000000FF5A5A00000008
```

Figure 4.4: Magic capabilities status bits on backdoor block of *Gen4* magic tag.

(21-22)-th: this 2-byte information enable/disable the shadow mode above described in the Section 4.4.2.

To enable shadow mode, set 21st and 22st bits to: **5A**; To disable shadow mode, set it to: **00** [7].

In Figure 4.5 is shown a block configuration where shadow mode is enabled.

```
[+] ----- GDM Gen4 Configuration -----
[+] 7AFF000000000000FF5A5A00000008
```

Figure 4.5: Shadow mode status bits on backdoor block of *Gen4* magic tag.

Chapter 5

Cloning Attack

Following the theoretical information provided in the previous chapters, this section presents the complete attack workflow used to clone a legitimate RFIF tag. The attack leverages the vulnerabilities discussed in Section 3.3.

The procedure is performed using a *Proxmark3* device, see Section 4.1, with its dedicated firmware. All the commands referenced in this section are in the official project wiki [32].

The cloning attack follows a precise scheme, inspired by standard attacker workflow [33], and is divided into four phases:

1. Recon Phase;
2. Data gathering Phase;
3. Weaponization Phase;
4. Delivery Phase.

A flow diagram, in Figure 5.1, illustrates the sequence of steps, providing a visual summary of the attack.

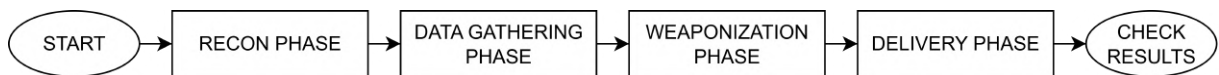


Figure 5.1: Attack flow diagram.

Once the clone has been created, the tag is tested on a real-world system to validate the success of the attack and confirm the full replication of the target's behavior.

All the script commands are sent to the *Proxmark* via command line using the *ProxSpace* terminal environment described in Section 4.2.

5.1 Attack Implementation

5.1.1 Recon Phase

As shown in Figure 5.1, the first step is the recognition phase. In this first step, we gathered some info about the actual data and how it is stored inside the tag in order to determine the most suitable attack strategy.

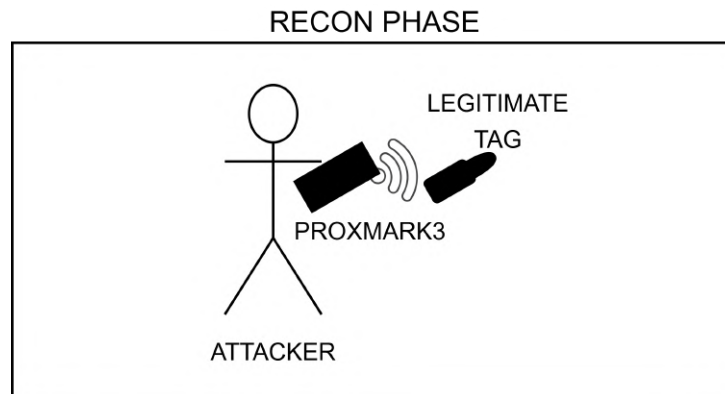


Figure 5.2: Visual representation of the attacker in the recon phase.

To do this, we start positioning the target legitimate tag on the Proxmark3 scan field and send the command

```
1 hf search
```

This command does a high frequency scan and performs a challenge-response authentication within all the tags inside the *Proxmark* field.

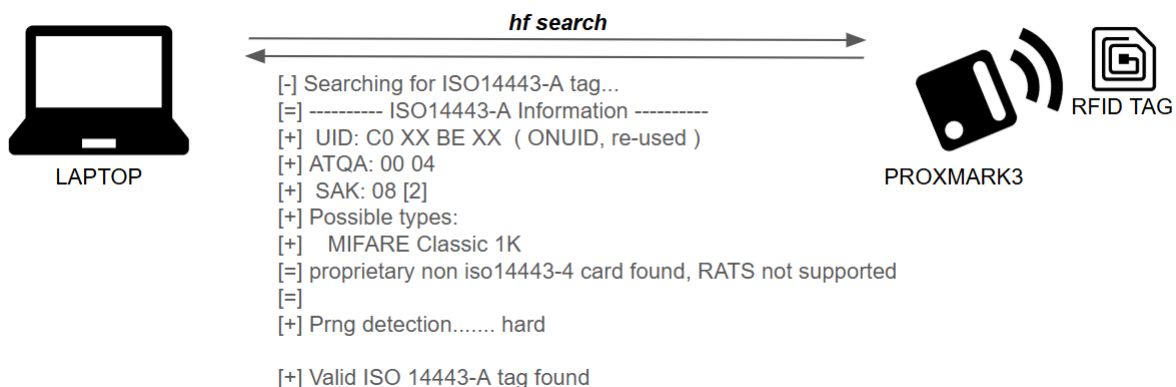


Figure 5.3: *hf search* response.

As we can see in Figure 5.3, the reader finds our legitimate tag and is capable of obtaining some precious information. It gives the *UID*, the *ATQA* and the *SAK* of the tag and its possible

standard type, that in our case is a *Mifare Classic 1K*.

The response tells us:

- **UID:** the tag's IDentifier.
- **Answer To Question A - ATQA:** gives us precious information on the authentication protocol mechanism, better explained in Section 3.2.1.
- **Selective Acknowledge - SAK:** to inform that the authentication process has been succesful.
- **MIFARE Standard 1K is used:** This last information is very important because it redirects us to the correct standard used and selects the correct algorithm to retrieve the secret keys of the target tag and be able to read the data stored inside.

5.1.2 Data gathering Phase

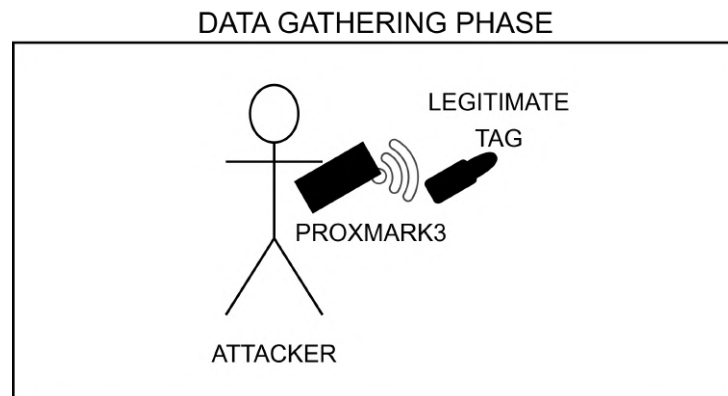


Figure 5.4: Visual representation of the attacker in the data gathering phase.

In Figure 5.4, a visual representation of the attacker position and tools used to perform this phase is shown.

Dictionary Attack

In the previous phase, it has been established that the target key uses the *MIFARE Classic 1K* standard.

Now, before trying more complex attacks, we check if some well-known keys are used to protect the memory access.

This type of attack, where a set of well-known or precomputed keys are tested, regardless of the result of the previously tested keys, is known as a *Dictionary attack*. To execute this basic attack, we launch the command

```
1 hf mf chk
```

The command uses:

- **hf**: tells the reader to operate in the high frequencies (13.56 MHz)
- **mf**: uses the correct standard(*MIFARE Classic 1K*)
- **chk**: runs a dictionary attack based on some commonly used secret keys. The key used comes with the *Proxmark* firmware.

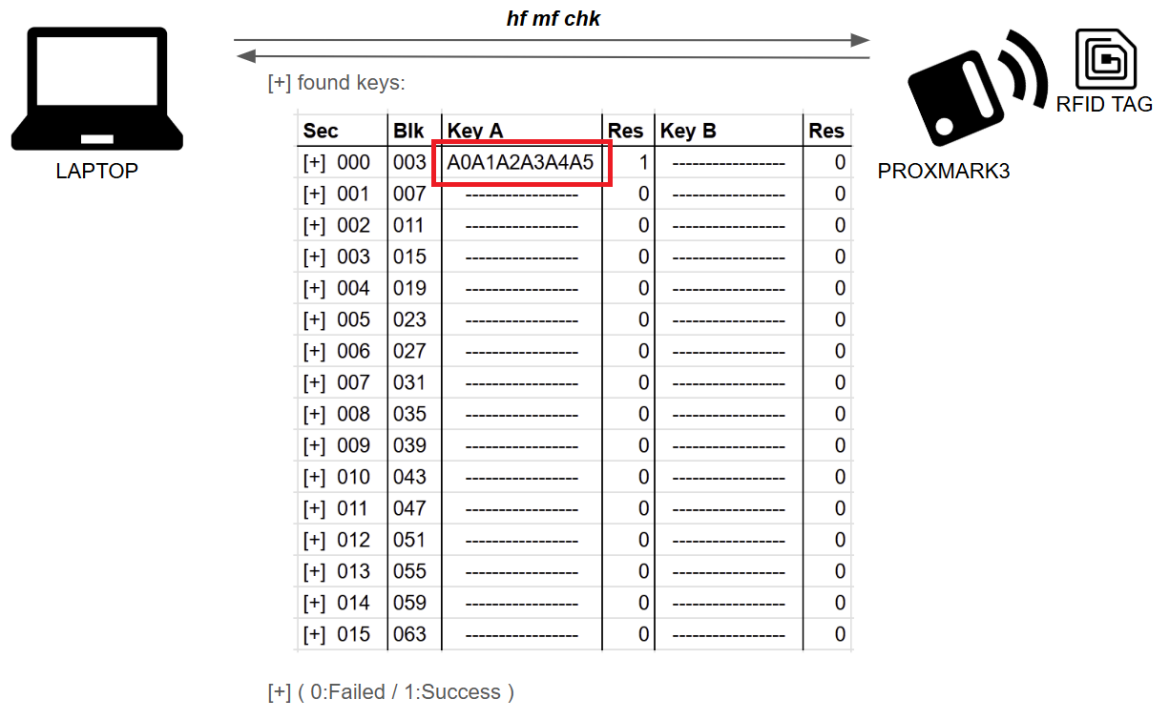


Figure 5.5: *hf mf chk* result.

The result of this first base approach has retrieved only the *Key A* of the manufacturer block, in Figure 5.5 outlined in red.

This can be misinterpreted as a failed attack, but it returns a very important starting point for the next brute-force approach.

With this information, we can now deploy a more elaborated attack that exploits the Nested Authentication mechanism explained in Section 3.3.2.

We deploy an *Nested/HardNested* attack that needs only one known key to obtain the others.

Nested/HardNested Attack

As we can see in Figure 5.5, the *Key A* of Sector 0 has been discovered, and now we use it in our attack.

To deploy the attack, we send the command

```
1 hf mf autopwn
```

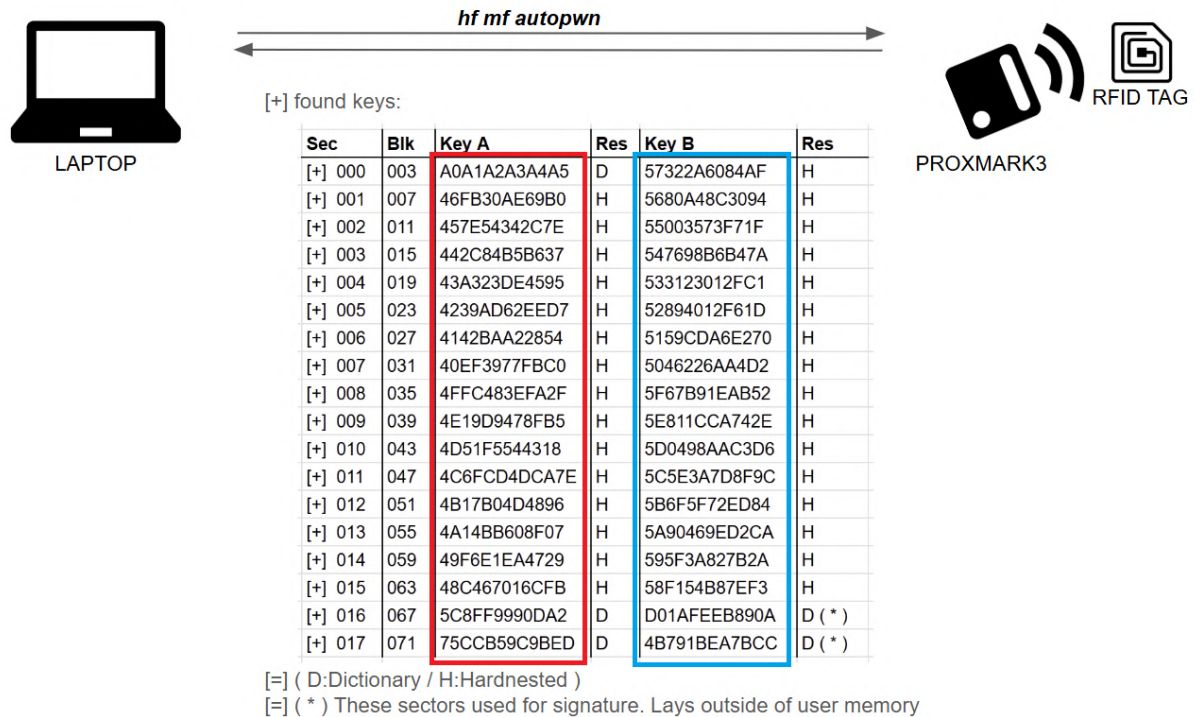


Figure 5.6: *hf mf autopwn* result.

The attack succeeded, and as we see in Figure 5.6 we are able to obtain all the secret keys. In Figure 5.6 in red are outlined all the *Keys A* and in blue all the *Keys B*.

A *.bin* file containing all the keys just obtained is created and stored in the *ProxSpace* environment that will be used later in the attack.

With the keys, we can now access all sectors of all blocks and obtain a complete dump of the target tag's memory.

Memory Dump

With all the keys, we can now obtain a full memory dump. We can achieve that by sending the command

```
1 hf mf dump
```

The command returns a complete dump of the memory of the target tag and recovers the key to access all the sectors from the *.bin* file previously stored. Also, the dump of the memory is stored in a *.bin* file.

In Figure 5.7, only a few bytes of the dump are displayed.

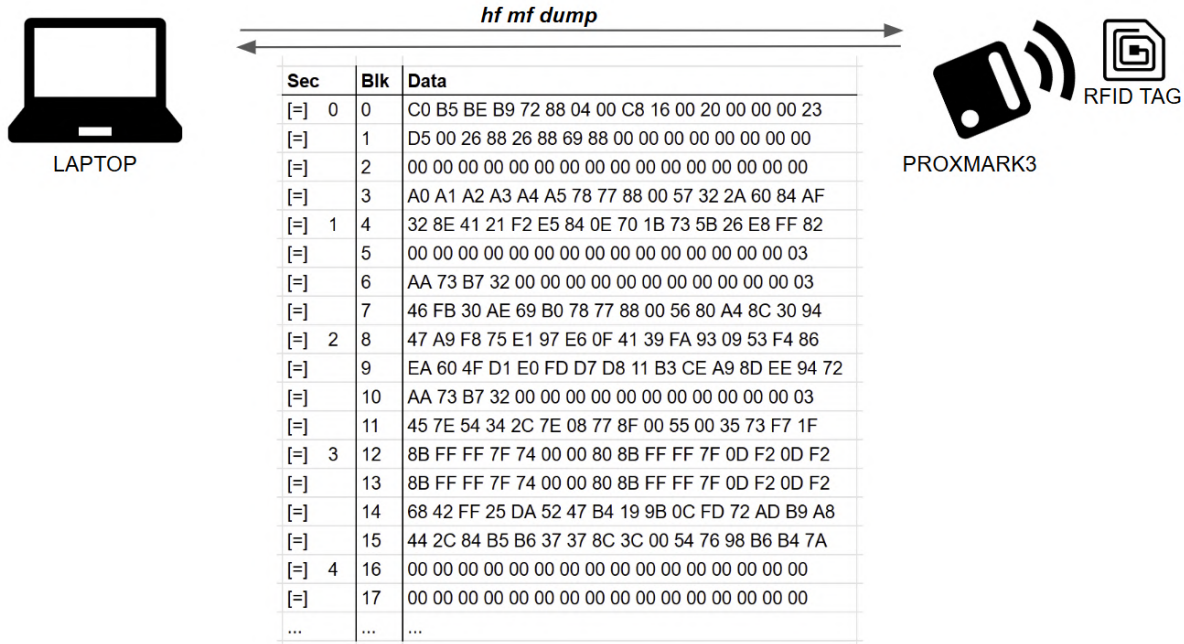


Figure 5.7: *hf mf dump* result.

5.1.3 Weaponization Phase

In this phase, we load on the magic tag the dump previously obtained. We have to follow different procedures for the two different types of tags used to perform the attack object of this thesis. As introduced in Section 4.4, the tags used are the first one developed *Gen1a*, and the newest, at the time of writing, *Gen4* [8].

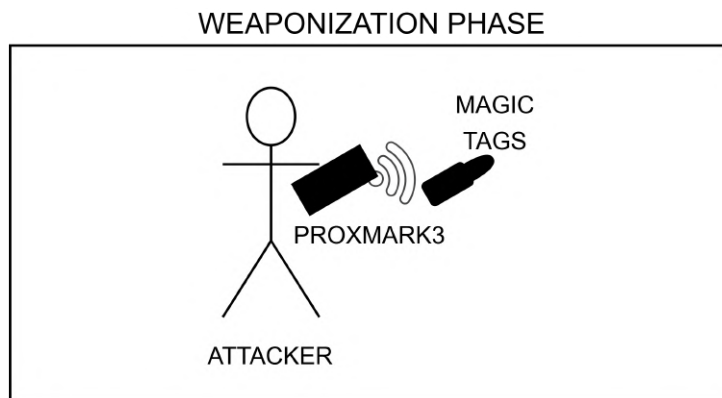


Figure 5.8: Visual representation of the attacker in the weaponization phase.

In Figure 5.8, a visual representation of the attacker and the devices used to complete the phase is shown.

Gen1a

For the first generation of tags, the procedure is easier. First, we have to identify the tags, and it's done with the same command used before. We position the *magic* tag in the *Proxmark* scan field and launch the command

```
1 hf search
```

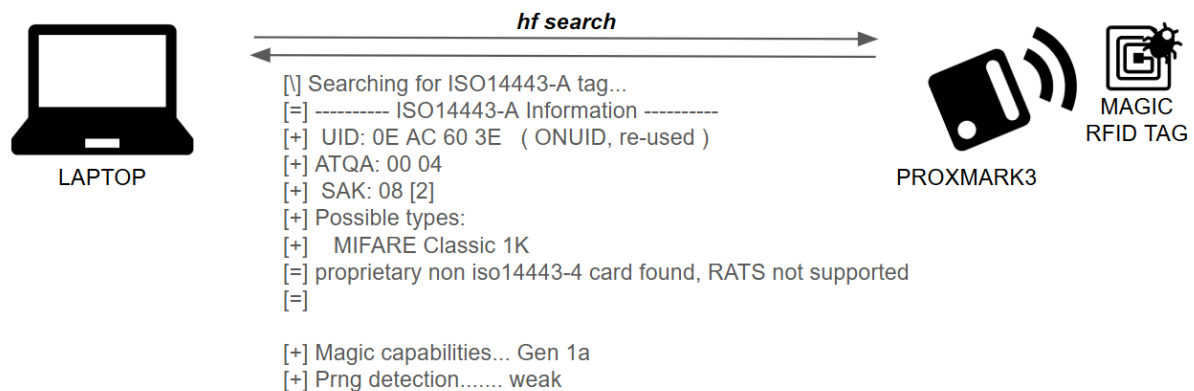


Figure 5.9: *hf mf search* result.

In this case, the scan correctly tells us that the tag has *Gen1a* magic capabilities.

We can now continue loading on the magic tag the memory dump of the attacked legit tag.

To complete the write in the magic tag's memory, we send the command

```
1 hf mf cload -f namefile-dump.bin
```

The command uses:

- **hf**: tells the reader to operate in the high frequencies (13.56 MHz)
- **mf**: uses the correct standard (*MIFARE Classic 1K*)
- **cload**: command available only for tags with magic capabilities and it permits the loading of a memory dump on the magic tag.
- **-f namefile-dump.bin**: a memory dump file has to be specified if the *cload* command is used.

To obtain a rapid dump of the memory, only in a magic tag though, we send the command

```
1 hf mf cview
```

Doing a rapid comparison between the stored dump file loaded and the effective content of the magic tag's memory after the write, we can confirm that the dump loading is successful.

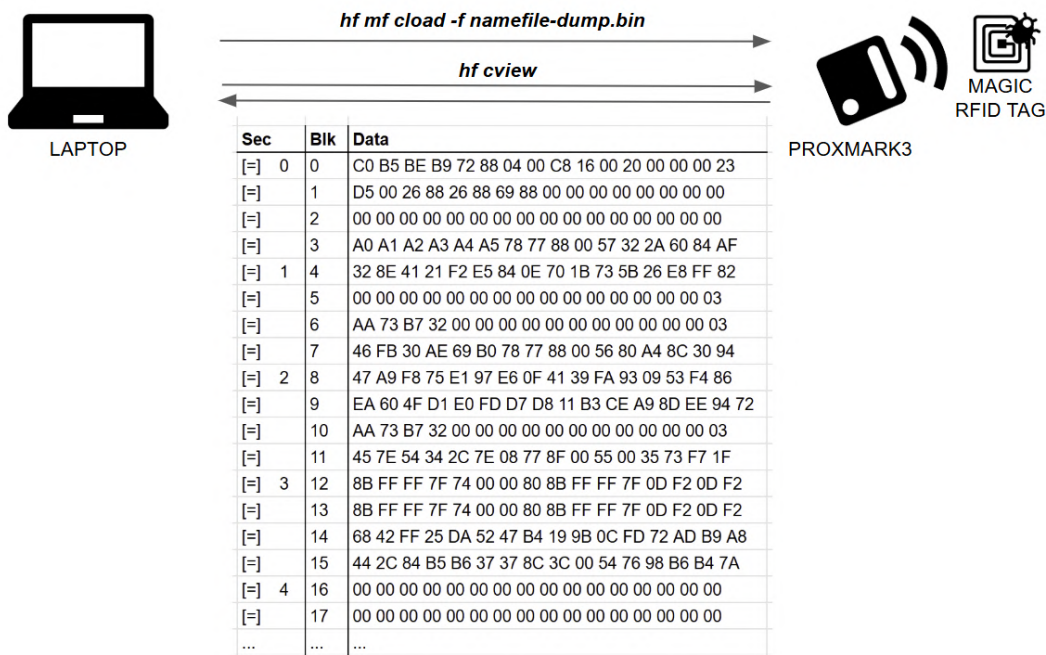


Figure 5.10: *hf mf cloud -f namefile-dump.bin* & *hf mf cview* result.

Gen4

Now we proceed to upload the dump of the legitimate tag in the *Gen4* magic tag.

Like what we have done for the *Gen1a* tag, we have to identify the tag first.

After positioning the magic tag in the reader's field, we newly send the command

```
1 hf mf search
```

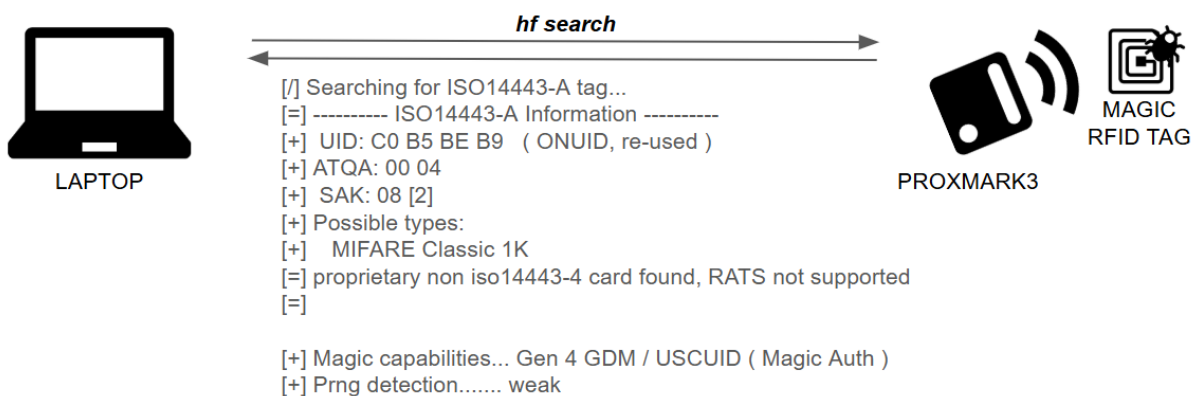


Figure 5.11: *hf mf search* result.

The scan confirms that the tag is of Gen 4.

This, new type of tag, behaves differently from the older version, and we have to enable the

magic capabilities first, that by default are disabled, in order to bypass reader detection of magic tags.

To enable the magic capabilities, described in Section 4.4, we send the command

```
1 hf mf gdmsetcfg --gdm -d 7AFF000000000000FF5A5A00000008
```

The command uses:

gdmsetcfg: permit to configure the backdoor block

-gdm: accessing the backdoor block using this flag, needed this flag given the current block configuration. More detail in Section 4.4.2

-d 7AFF000000000000FF5A5A00000008: we set the backdoor block to this value, enabling the magic capabilities and the shadow mode [7].

To check the correct setting up of the backdoor block, we send the command

```
1 hf mf gdmcfg
```

The command prints the current status of the backdoor block.

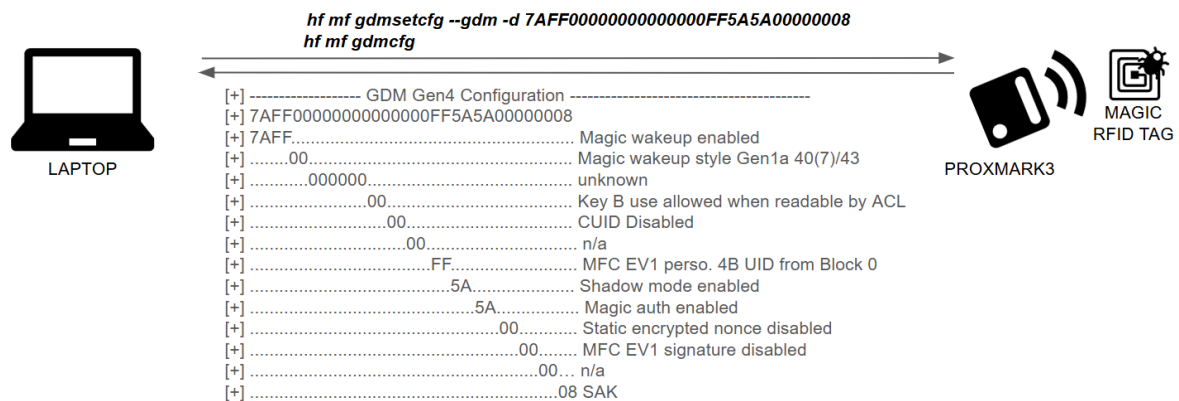


Figure 5.12: *Backdoor memory* setup command.

Now we are sure that the magic capabilities are enabled, we can write in the magic tag's memory and load the dump of the legitimate tag.

Like before, we send the command

```
1 hf mf cload -f namefile-dump.bin
```

After loading the dump on the magic tag, we restore the backdoor block.

As described in Section 4.4.2, we restore the backdoor block to its previous state to disable magic capabilities in order to not be detectable as *Magic tag* by the reader. We send the command

```
1 hf mf gdmsetcfg -d 8500000000000000FF5A5A00000008
```

Referring to Section 4.4.2, this block value disables the magic capabilities.

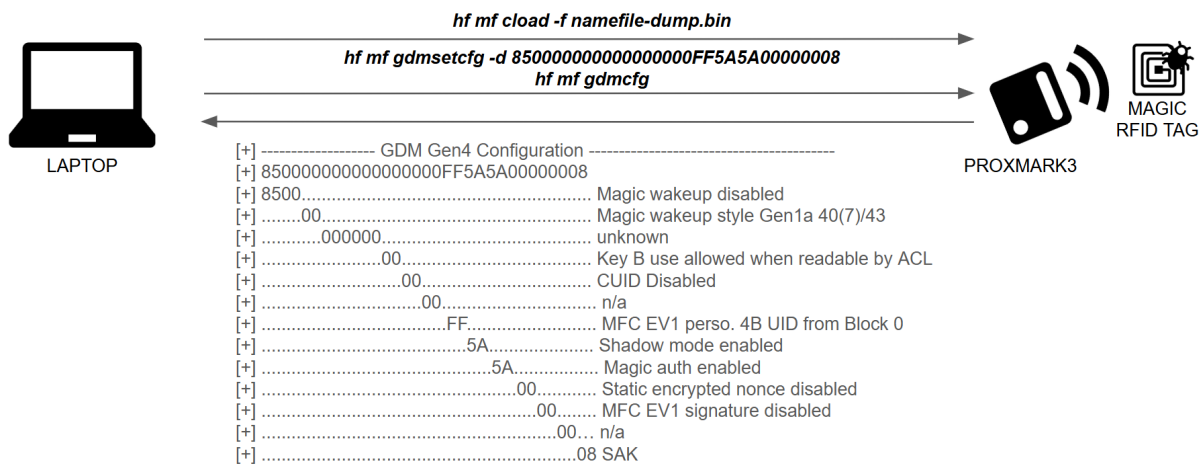


Figure 5.13: *Dump load & Backdoor memory reset.*

Now, we do a double check; we send the command.

```
1 hf mf cview
```

This command, like said before, is available only for magic tags, but trying to execute on our *Gen4* tag it returns an error. The command failed because our tag, with the current backdoor block's configuration, is not seen as a magic tag but as a legit one. This is the feature that permits us to succeed in the attack.

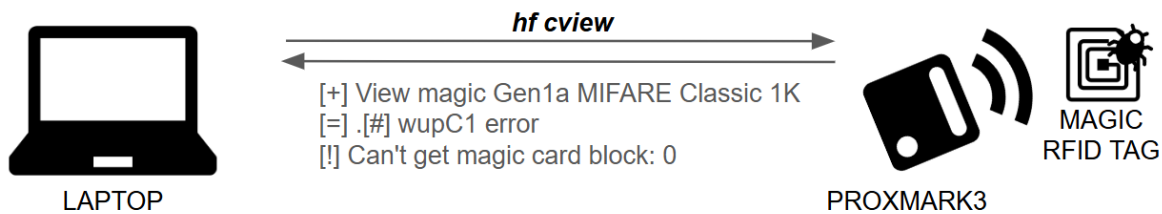


Figure 5.14: *hf mf cview* error.

5.1.4 Delivery Phase

The final stage of our attack involves deployment and validation of the cloned RFID magic tags in a real-world scenario. In this phase, we verify whether the attack was successful by attempting to use the cloned tags in place of the proprietary legit tag. The target system is a vending machine adopting *MIFARE Classic* as standard.

This stage essentially tests if the system accepts the magic tags as legit ones, and if the magic tags behave correctly.

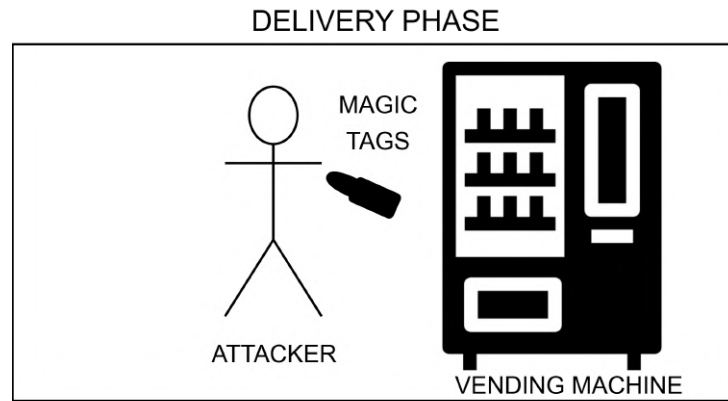
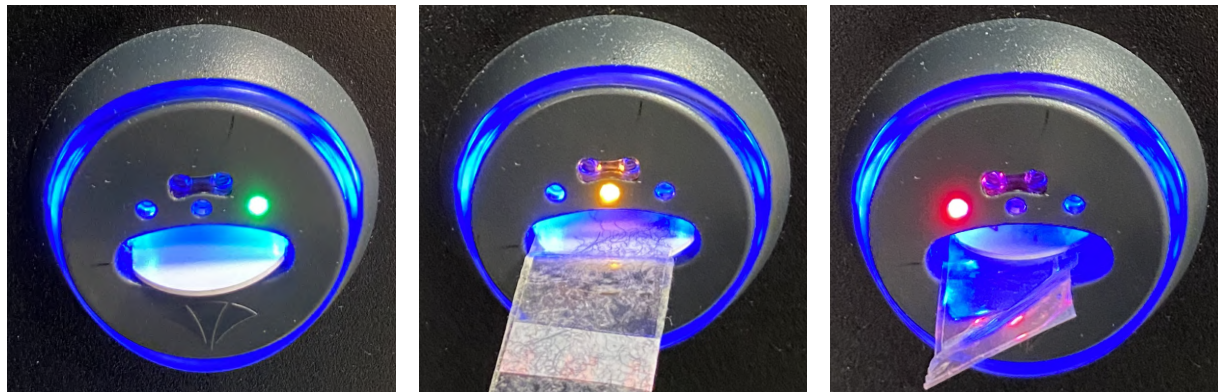


Figure 5.15: Visual representation of the attacker in the delivery phase.

As shown in Figure 5.15, this is the first time that the attacker has to interact with the vending machine; in the previous phases, it is not required to interact with the vending machine.

The reader, the vending machine, has three status lights indicating the status of the communication with the RFID tags:

- **Green light:** the reader is ready to establish a new communication with a RFID tag.
- **Yellow light:** the reader has established an ongoing communication with a RFID tag.
- **Red light:** the reader has established a communication with an RFID tag, but the data stored inside it is corrupted or has been tampered.



(a) Green vending machine's status light.

(b) Yellow vending machine's status light.

(c) Red vending machine's status light.

Figure 5.16: All vending machine's status lights.

Gen1a delivery

We start the testing with the *Gen1a* magic tag. To test the tag, we follow the same procedures as using a legit tag; we insert the cloned *Gen1a* tag in the reader and wait until it detects it (*a*

yellow status light appear on the reader, meaning communication established) and shows at least the balance stored. After this, we can proceed to do a test transaction and verify the correct behavior.

After inserting our cloned magic *Gen1a* tag in the vending machine, it fails to detect it; the status light is green, see Figure 5.19, so the reader is ready to accept new RFID tags. If some errors would have occurred in the writing process resulting in corrupted memory data, the status light on the machine would have been red. See Figure 5.17.



Figure 5.17: Gen1a magic tag with corrupted memory deployed.

To better understand what happened, we do a dump of the magic tag’s memory and analyze it. We send the command

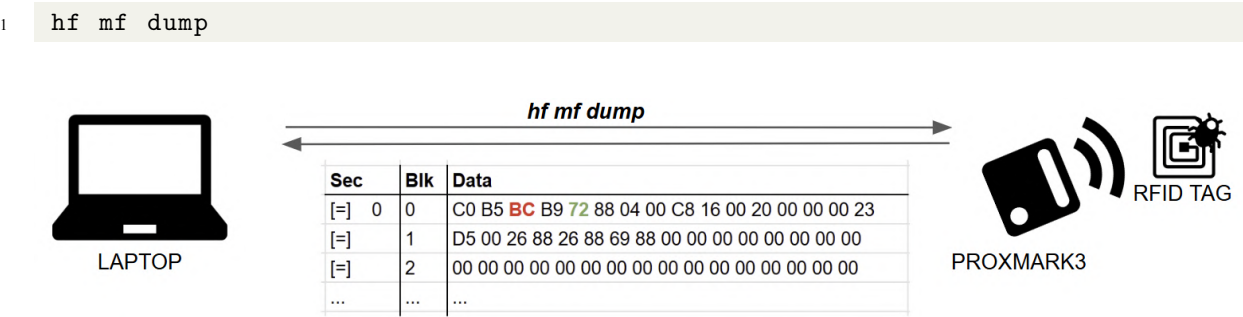


Figure 5.18: *hf mf dump* after the delivery phase.

The first thing that catches the attention is that the *UID* of the magic tag has been modified since our write. We further analyze this aspect in Section 6.1.

We try to restore the memory to the previous correct state; to do this, we redo the steps starting from the *Weaponization phase* 5.1.3. Now that the tag is newly correct and ready, we try again. Following the same procedures, we test the tag, but the result is the same. With the *Gen1a* magic tag, the attack failed. See Figure 5.19. The reason for the failure is discussed in Section 6.1.



Figure 5.19: Gen1a magic tag with poisoned UID deployed.

Gen4 testing

We repeat the same test now using the *Gen4* magic tag. In this case, the reader status light turns yellow, and the balance is correctly shown. A step forward has been made since the previous tests with the *Gen1a* tag.

We proceed with the test by trying to complete a transaction, and all the process goes correctly, like using a legit proprietary tag.

After the transaction is complete and the credit is correctly deducted, we remove the magic tag from the reader's field, let the tag discharge, and restore its memory. See the detailed procedures in Section 2.1.1, 4.4.2.

We now check again the credit value and we state that it's as if the transaction has never happened; the credit is equal to the one before the transaction. See Figure 5.20.

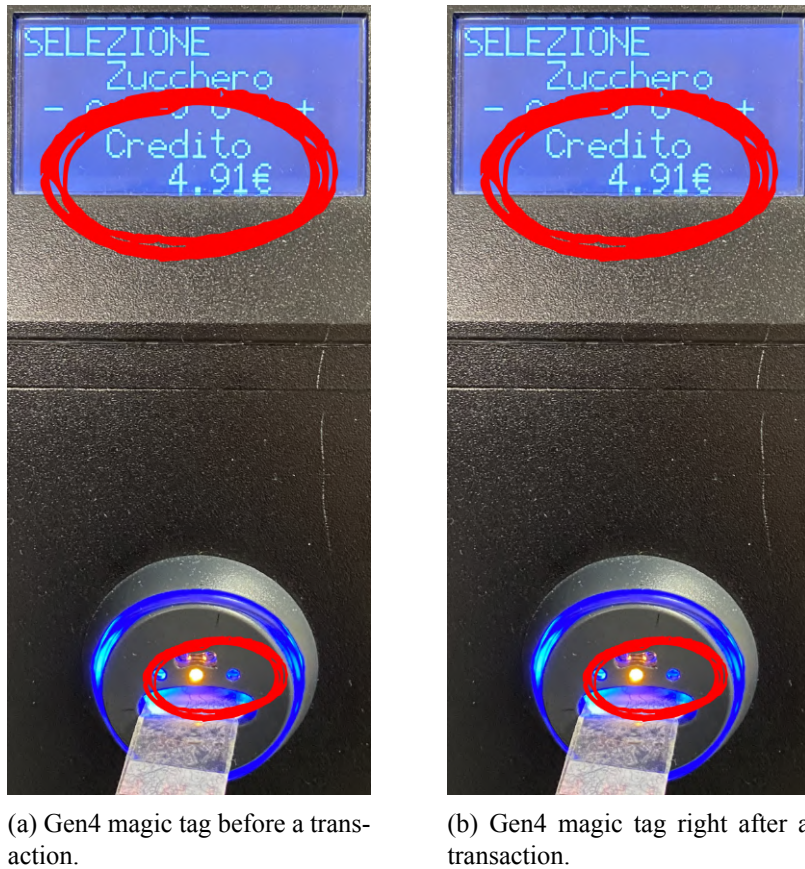


Figure 5.20: Gen4 magic tag before and after a transaction.

5.2 Motivation behinds the attack

After the technical and detailed procedures described in the above paragraphs to perform the attack, we further analyze what can push an attacker to target a legacy system using *MIFARE Classic*.

The motivations behind the attack may vary; below, some are proposed:

- *Direct financial gain*: In payment-based system such as vending machines, a cloned tag can be used to obtain products without paying. If undetected this can lead to significant financial losses.
- *Scalability of fraud*: Once a valid tag is cloned, it can be duplicated and distributed, leading to a large-scale fraud. In past real-world cases, cloned public transport tickets have been sold online and used collectively [34].

- *Anonymity and low risk:* Many *MIFARE Classic*-based systems does not associate tags with specific people or account, reducing the risk of detection and legal consequences.
- *Level of skill required:* Devices like the Proxmark3 [28], along with the well-documented manual [32] and the availability of the necessary tools, provide a lower level of skill required.
- *Reputation or experimentation:* In security communities, success in exploiting a vulnerable system can be a way to build credibility or contribute to the research field.

The following section provides the results of the attack and also analyzes the successful and unsuccessful attacks performed. It also provides an overview of the complexity and impact of the attack in a real-world implementation, followed by strategies to overcome the vulnerabilities.

Chapter 6

Security Analysis and Mitigations

6.1 Results

The delivered attack has succeeded using the *Gen4* tag, but has failed with the *Gen1a* magic tag.

We have completed a **cloning attack** in both generations of tags, and with the *Gen4* tag we succeeded in performing a **double-spending attack** and **replay attack**.

To understand why *Gen1a* tag attack failed, we have to take a step back and analyze how the manufacturer block on a *Mifare Classic 1K* chipset is organized.

4 + 1 Bytes	2 Bytes	9 Bytes
UID + BCC	SAK + ATAQ	Manufacturer Data

Figure 6.1: Manufacturer Block on Mifare Classic 1K.

In Figure 6.1 is shown the detailed configuration of a manufacturer block with a 4-Byte NUID.

To the standard authentication process of *MIFARE Classic* standard, described in Section 3.2.1, the attacked system adopts an additional mechanism to detect if tags are legit or not. The machine tries to poison the UID of the tags by changing a byte of it to a random value. If the machine succeeds in the poisoning attack, the manufacturer block structure of the magic tag is invalidated, due to the previously described mechanism of *BCC* checksum check. After invalidating the *BCC* byte, the *MIFARE Classic* authentication mechanism fails.

This happens because in a legit tag, the manufacturer block is a *read-only* block, so it cannot be tampered with invalid data, and it cannot be affected by this defense strategy.

This defense strategy works against *Gen1a* clone tags, but with the new generation of tags tested, our attacks prove that it is possible to bypass this security mechanism.

6.2 Attack Complexity

While the attack on *MIFARE Classic* tags using a *Gen4* magic tag requires technical knowledge of MIFARE UID and memory structure, nested authentication exploit, and detailed knowledge of the functionalities of the magic tags used, it is not beyond the capabilities of moderately skilled attackers. Devices like the Proxmark3 [28], combined with public papers on the topic [27][6], significantly lower the difficulty of the attack.

The cloning attack is the easiest and most rapid attack to achieve, but for the same reasons, it is also the most dangerous and potentially the most damaging in financial terms.

Once a target system is identified, the full cloning attack, including all phases, see Figure 5.1, such as reconnaissance, data gathering, weaponization, and delivery, is achievable within 15 to 60 minutes, depending on the number of keys to brute-force, the machine's computational speed, and the attacker's level of expertise and knowledge.

6.3 Economic Impact

From an economic perspective, each successful clone can represent a tangible financial loss for the attacked system. In vending systems, assuming each cloned tag is used to illegitimately purchase items, and assuming, per single use, a transaction ranging from 0.50 € for a coffee to 2 € for a sandwich, a single tag can cause cumulative losses of 50 € - 150 € per month if undetected, especially if the tag is used in areas with high affluence. To denote also that, if an attacker start selling cloned tags to other people, spreading cloned tags, the total damage can reach thousands of euros per month.

Furthermore, as previously introduced, the cloning process does not require to be done near or using the vending machine. So, potentially, an attacker could produce a significant number of cloned tags without ever having been in contact with the vending system .

6.4 Possible Mitigation Strategies

To prevent this attack, and the potential economic losses described in Section 6.3, it is good practice to implement some security mechanisms. In this section, a couple of strategies to manage

the attack are proposed.

Update tag chipset

A possible solution can be the update of the tag's chipset.

The *Mifare Classic 1K* standard is, as we can see, still functioning but obsolete and deprecated. The new *Mifare DESFire Light* standard can be an optimal substitute and is proposed by NXP as equivalent to *Mifare Classic 1K* functionality.

It offers *AES* encryption, a new Leakage Resilient Primitive (LRP) for secure messaging, employs mutual authentication between the card and reader, incorporates hardware security features, and is certified according to Common Criteria EAL4 [35].

This is the most secure and long-term solution, but it requires significant costs, concerning:

- Updates of the reader firmware and hardware.
- Updates the software to support the new protocol.
- Distribute to all the users the new updated tags.

For large systems, (e.g., University), this can be a lot expensive, and also the retro-compatibility of *MIFARE DESFire* with *MIFARE Classic* is not guaranteed [36].

Remote Storage of the balance

One option is to avoid storing any credit on the tag itself and keep only the *UID* there. All balance information would reside in an external database.

The vending machine already has a module for internet connectivity and credit-card payments; this existing hardware can be leveraged to implement remote balance checks, making it much harder to tamper with stored credit.

In this scenario, a cloned tag gains no advantage, since the true balance never lives on the card.

In this option, the main challenge is to have low latency when performing a transaction and ensure the availability of the connection with the backend server. To achieve this, it is required:

- Develop secure API - Application Programming Interface, set of rules or protocols that enables software applications to communicate with each other to exchange data [37].
- Secure and reliable database or cloud infrastructure.
- A local cached fallback copy of the data used to continue providing the service also offline, accepting the risk of potential attacks [38].

Signature and Counter validation

On the legit tag, a monotonic counter is stored and updated at each transaction. That counter, together with the *UID* and other data, is used to compute a cryptographic signature that the

vending machine verifies before approving the operation. By itself, however, this scheme does not stop a *double-spending* attack: an attacker could read the tag's signature, perform a purchase, and then replay the same valid signature.

To fix this, the vending machine must also store the last-known counter value for each UID. On each transaction, the vending machine reads the counter stored on the tag, verifies the signature using its secret key, and then checks that the counter on the machine-side is the same as the one on the tag.

If so, it approves the transaction and updates the stored counter; otherwise, it rejects the transaction.

In addition, with the counter written on the tag, a fresh signature must be provided.

Because the attacker cannot forge that signature, it cannot be tampered with.

In this way, even if someone clones the tag, the stored counter on the machine-side prevents the same signature from being used twice, blocking double spending.

This mechanism is highly effective against replay and double-spending attacks; in fact, it is adopted in the Europay, Mastercard, and Visa - EMV integrated circuit [39].

The costs of this option rely on the system complexity, as described in [39], it requires the secure storage of master keys, a session-key per each tag/chip, and a cryptographic coprocessor to perform the EMV functions.

Despite these, it is the most practical solution, to obtain strong replay and double-spending attack protection without migrating to *MIFARE DESFire* chips [35].

Chapter 7

Conclusion

The study confirms that Mifare Classic 1K RFID tags, despite their widespread usage in various commercial infrastructures, continue to be vulnerable and insecure.

Through a detailed theoretical examination of their internal structure, cryptographic mechanism and vulnerabilities, it was possible to execute a complete tag cloning attack, using tools that are available to everyone and relatively at a low cost, such as the RFID reader *Proxmark3 RDV4.01* and *Gen4* magic tags.

The attack was divided into sequential phases, such as recognition, data gathering, weaponization, and delivery, succeeding in cloning a legitimate *MIFARE Classic 1k* tag using a *Gen4* magic tag. The cloned tag not only was able to present itself at the reader as an authentic tag, but also allowed the execution of double-spending and replay attacks, simulating a real-world fraud scenario in a vending machine system. In contrast, older versions of magic tags, such as the tested *Gen1a*, were blocked by the reader defense strategy, based on UID poisoning and consequent UID validation.

From a practical point of view, the concerning thing is the feasibility of such an attack. The complete process of cloning a legitimate tag can be executed within 15 up to 60 minutes depending on the attacker's expertise and the hardware availability. Another concerning point is the stealth of the attack, because an attacker can complete the cloning process without the need to access or interact with the vending machine.

Another important aspect is the economic one. A single cloned tag can lead to economic losses ranging from a few euros to hundreds per month. If cloned tags start to spread out, especially in environments with a lot of people like universities, the losses can soar exponentially if no security mechanisms are implemented.

In this thesis, several strategies have been proposed and analyzed: Updating the chip's hardware to more secure ones, like *MIFARE DESFire Ligh*t. This is the most secure solution, but it involves significant costs related to the renewal of the system infras-

tructure.

Another proposed mitigation is the remote storage of the balance so the RFID tag carries only an identifier, but this approach requires a reliable backend structure and databases, secure APIs, and a constant connection.

The last mitigation method proposed is currently used by the EMV circuit, used in credit cards, and prevents replay attacks by verifying that each transaction is unique and verifiable, but this method requires additional cryptographic components and the secure storage of session keys.

In conclusion, this work emphasizes the importance of not underestimating the risks associated with using RFID legacy systems, particularly in financial transactions. Through practical experimentation, this thesis demonstrates that real-world attacks are not just theoretically possible but, with modest effort, put into reality. As the RFID technology continues to spread and evolve, proactive security strategies must be implemented to ensure safety and security.

Bibliography

- [1] K. Finkenzeller, *RFID Handbook: Fundamentals and Applications in Contactless Smart Cards, Radio Frequency Identification and Near-Field Communication*, 3rd ed. Chichester, UK: Wiley, 2010, isbn: 978-0470695067.
- [2] E. Ilie-Zudor, Z. Kemény, P. Egri, and L. Monostori, “The rfid technology and its current application,” *The Modern Information Technology in the Innovation Processes of the Industrial Enterprises-MITIP*, pp. 29–36, 2006.
- [3] R. Want, “An introduction to rfid technology,” *IEEE Pervasive Computing*, 2006.
- [4] S. Ahuja and P. Potti, “An introduction to rfid technology,” *Communications and Network*, pp. 183–186, 2010.
- [5] NXP Semiconductors, *MIFARE Classic | NXP Semiconductors*, https://www.nxp.com/products/rfid-nfc/mifare-hf/mifare-classic:MC_41863, Accessed: 2025-06-30, 2025.
- [6] K. Nohl, D. Evans, Starbug, and H. Plötz, “Reverse-engineering a cryptographic rfid tag,” in *Proceedings of the USENIX Security Symposium*, USENIX Association, San Jose, CA, 2008. [Online]. Available: https://www.usenix.org/legacy/event/sec08/tech/full_papers/nohl/nohl.pdf.
- [7] myshamye, *Proxmark3/doc/magic_cards_notes.md at master · rfidresearchgroup/proxmark3 · github*, 2025. [Online]. Available: https://github.com/RfidResearchGroup/proxmark3/blob/master/doc/magic_cards_notes.md.
- [8] Lab401, *Know your magic cards*, Lab401 Academy blog, Accessed July 2, 2025. Navigating types and features of ”Magic” RFID cards, 2019. [Online]. Available: <https://lab401.com/blogs/academy/know-your-magic-cards>.
- [9] F. Dehousse and T. Zgajewski, “What is rfid?” Egmont Institute, Tech. Rep., 2009.
- [10] D. M. Dobkin, *The RF in RFID: UHF RFID in Practice*, 2nd. Boston, MA: Newnes, 2012, p. 540, isbn: 9780123948304.

- [11] *ISO/IEC 14443-2:2016 – identification cards – contactless integrated circuit cards – proximity cards – part 2: Radio frequency power and signal interface*, <https://www.iso.org/standard/73596.html>, See Clauses 5–7 for inductive coupling and load modulation specifications. International Organization for Standardization, 2016.
- [12] P. Kitsos and Y. Zhang, Eds., *RFID Security: Techniques, Protocols and System-On-Chip Design*. New York: Springer, 2008, isbn: 978-0-387-76480-1. doi: 10.1007/978-0-387-76481-8.
- [13] B. Crispo and A. S. Tanenbaum, “Rfid malware: Truth vs. myth,” *IEEE Security and Privacy Magazine*, 2006.
- [14] M. R. Rieback, B. Crispo, and A. S. Tanenbaum, “Is your cat infected with a computer virus?” In *Proceedings of the 2006 IEEE Symposium on Security and Privacy (S&P’06)*, IEEE, 2006, pp. 169–183. doi: 10.1109/SP.2006.30. [Online]. Available: <https://www.cs.vu.nl/~ast/Publications/Papers/percom-2006.pdf>.
- [15] N. Semiconductors, *Mifare classic ev1 1k — mainstream contactless smart card ic for fast and easy solution development*, MF1S50YYX_V1, Rev. 3.2, 2018. [Online]. Available: https://www.nxp.com/docs/en/data-sheet/MF1S50YYX_V1.pdf.
- [16] F. Garcia, G. Gans, R. Muijrrers, *et al.*, “Dismantling mifare classic,” vol. 5283, Oct. 2008, pp. 97–114, isbn: 978-3-540-88312-8. doi: 10.1007/978-3-540-88313-5_7.
- [17] B. Fredriksson, “A case study in smartcard security, analysing mifare classic rev. 1,” Monash Univeristy, Tech. Rep., 2016.
- [18] F. Masoodi, S. Alam, and M. U. Bokhari, “An analysis of linear feedback shift registers in stream ciphers,” *International Journal of Computer Applications*, vol. 46, no. 17, May 2012, issn: 0975-8887.
- [19] A. Juels, “Rfid security and privacy: A research survey,” *Selected Areas in Communications, IEEE Journal on*, vol. 24, pp. 381–394, Mar. 2006. doi: 10.1109/JSAC.2005.861395.
- [20] A. Juels, R. L. Rivest, and M. Szydlo, “The blocker tag: Selective blocking of rfid tags for consumer privacy,” in *Proceedings of the 10th ACM conference on Computer and communications security*, ACM, 2003, pp. 103–111.
- [21] S. A. Weis, S. E. Sarma, R. L. Rivest, and D. W. Engels, “Security and privacy aspects of low-cost radio frequency identification systems,” in *Security in Pervasive Computing*, Springer, 2004, pp. 201–212.

- [22] M. Nieves, K. Dempsey, and V. Y. Pillitteri, “An introduction to information security,” National Institute of Standards and Technology (NIST), NIST Special Publication 800-12 Rev. 1, 2017, Accessed content on page 77. doi: 10.6028/NIST.SP.800-12r1. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-12r1>.
- [23] *ISO/IEC 14443-3:2016 identification cards – contactless integrated circuit cards – proximity cards – part 3: Initialization and anti-collision*, Standard, See Section 6.3 for parity bit usage, Geneva, Switzerland: International Organization for Standardization, 2016.
- [24] *ISO/IEC 14443-4: Identification cards — Contactless integrated circuit cards — Proximity cards — Part 4: Transmission protocol*, Available from <https://www.iso.org/standard/73596.html>, Geneva, Switzerland: International Organization for Standardization, 2018.
- [25] NXP Semiconductors, “AN10834: MIFARE ISO/IEC 14443 PICC Selection,” NXP Semiconductors, Application Note AN10834, version Rev. 4.2, 2021, Company Public. [Online]. Available: <https://www.nxp.com/docs/en/application-note/AN10834.pdf>.
- [26] K. Nohl, *Cryptanalysis of crypto-1*, Technical report / presentation, <https://fragdenstaat.de/files/foi/580223/Mifare.Cryptanalysis.pdf>, 2008.
- [27] F. D. Garcia, P. van Rossum, R. Verdult, and R. Wichers Schreur, “Wirelessly pickpocketing a mifare classic card,” in *2009 IEEE Symposium on Security and Privacy*, IEEE, 2009, pp. 3–15.
- [28] *Proxmark 3 rdv4.01 — lab401*, 2025. [Online]. Available: <https://lab401.com/en-it/products/proxmark-3-rdv4>.
- [29] Gator96100, *Github - gator96100/proxspace: Proxmark iii development environment for windows*, 2023. [Online]. Available: <https://github.com/Gator96100/ProxSpace>.
- [30] *Mifare classic® compatible 1k uid modifiable – pack of 5*, Lab401 product page, Accessed July 2, 2025; 1 KB memory, 4 byte UID, UID modifiable multiple times, sold in packs of 5, Lab401 / ÉTOILE401 SAS, Jul. 2025. [Online]. Available: <https://lab401.com/it-it/collections/rfid-badges-hf/products/mifare-compatible-1k-uid-modifiable-pack-of-5>.
- [31] *Mifare classic® compatible 1k one time write uid modifiable coin tag (gen2a)*, Lab401 product page, Accessed July 2, 2025; features Gen2a one time write UID and 1K memory size, Lab401 / ÉTOILE401 SAS, Jul. 2025. [Online]. Available: <https://lab401.com/it-it/collections/rfid-badges-hf/products/mifare-classic-compatible-1k-one-time-write-uid-modifiable-coin-tag-gen2a>.

- [32] Proxmark / proxmark3 contributors, *commands · Proxmark/proxmark3 Wiki*, GitHub Wiki, Last edited March 23, 2021; accessed July 2, 2025, Mar. 2021. [Online]. Available: <https://github.com/Proxmark/proxmark3/wiki/commands>.
- [33] E. M. Hutchins, M. J. Cloppert, and R. M. Amin, “Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains,” Lockheed Martin Corporation, Technical Report, 2011.
- [34] F. D. Garcia, R. Verdult, and R. Wichers Schreur, “Lockpickers’ nightmare: Lessons from real-world attacks on transport systems,” in *Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security (CCS)*, ACM, 2013, pp. 929–940.
- [35] *Mifare® desfire — nxp semiconductors*, <https://www.nxp.com/products/rfid-nfc/mifare-hf/mifare-desfire>, Accessed 2025-07-15, 2025.
- [36] N. Semiconductors, *Mifare desfire light features and hints*, AN12343, Rev. 1.1, 2020. [Online]. Available: <https://www.nxp.com/docs/en/application-note/AN12343.pdf>.
- [37] M. Goodwin, *What is an api (application programming interface)?* <https://www.ibm.com/think/topics/api>, Accessed 2025-07-15, Apr. 2024.
- [38] OWASP Cheat Sheet Series Team, *Owasp cheat sheet series*, <https://cheatsheetseries.owasp.org/index.html>, Accessed 2025-07-15; licensed under CC BY-SA 4.0, n.d.
- [39] *EMV Integrated Circuit Card Specifications*, <https://www.ibm.com/docs/en/zos/3.1.0?topic=cryptography-emv-integrated-circuit-card-specifications>, Accessed 2025-07-15; corresponds to z/OS 3.1 documentation, 2025.